In [171…
```
import pandas as pd
import numpy as np
```

Part 1. Data Validation, Cleaning & Feature Engineering

Objective

Prepare Orders dataset for further KPI and profitability analysis.

Tasks

1. Validate structure and data types
2. Check data quality (missing values, duplicates)
3. Engineer shipping duration feature
4. Detect data anomalies

Data Loading

In [128…
```
file_path = "CV_Sample - Superstore - 2025.xls"

orders = pd.read_excel(file_path, sheet_name="Orders", parse_dates=
people = pd.read_excel(file_path, sheet_name="People",parse_dates=F
returns = pd.read_excel(file_path, sheet_name="Returns",parse_dates
```

Initial Overview

In [113…
```
orders.shape
```

Out[113…
```
(10194, 21)
```

In [130…
```
orders.dtypes
```

Out[130…

```
Row ID                     int64
Order ID                  object
Order Date        datetime64[ns]
Ship Date         datetime64[ns]
Ship Mode                 object
Customer ID               object
Customer Name             object
Segment                   object
Country/Region            object
City                      object
State/Province            object
Postal Code               object
Region                    object
Product ID                object
Category                  object
Sub-Category              object
Product Name              object
Sales                    float64
Quantity                   int64
Discount                 float64
Profit                   float64
dtype: object
```

Dataset contains 10194 rows and 21 columns. Date columns are stored correctly as datetime64.

Data Quality Checks

Missing Values

In [156…

```
orders.isna().sum()
```

Out[156…

```
Row ID            0
Order ID          0
Order Date        0
Ship Date         0
Ship Mode         0
Customer ID       0
Customer Name     0
Segment           0
Country/Region    0
City              0
State/Province    0
Postal Code       0
Region            0
Product ID        0
Category          0
Sub-Category      0
Product Name      0
Sales             0
Quantity          0
Discount          0
Profit            0
dtype: int64
```

No missing values found

Duplicate Key Check

In [180…

```
orders.duplicated().sum()
```

Out[180…

```
0
```

No duplicate values found

Numerical Range Validation

In [186…

```
numeric_cols=['Sales','Quantity','Discount','Profit']
orders[numeric_cols].describe()
```

Out[186…

	Sales	Quantity	Discount	Profit
count	10194.000000	10194.000000	10194.000000	10194.000000
mean	228.225854	3.791838	0.155385	28.673417
std	619.906839	2.228317	0.206249	232.465115
min	0.444000	1.000000	0.000000	-6599.978000
25%	17.220000	2.000000	0.000000	1.760800
50%	53.910000	3.000000	0.200000	8.690000
75%	209.500000	5.000000	0.200000	29.297925
max	22638.480000	14.000000	0.800000	8399.976000

In [190…

```
invalid_sales = (orders["Sales"] < 0).sum()
invalid_quantity = (orders["Quantity"] <= 0).sum()
invalid_discount_low = (orders["Discount"] < 0).sum()
invalid_discount_high = (orders["Discount"] > 1).sum()
negative_profit_count = (orders["Profit"] < 0).sum()
negative_profit_share = negative_profit_count / len(orders)
```

In [210…

```
range_validation_summary = pd.Series({
    "Negative Sales": invalid_sales,
    "Negative Quantity": invalid_quantity,
    "Negative Discount": invalid_discount_low,
    "Too high Discount": invalid_discount_high,
    "Negative Profit (count)": negative_profit_count,
    "Negative Profit (share)": round(negative_profit_share,3)
})
range_validation_summary
```

Out[210…

```
Negative Sales                 0.000
Negative Quantity              0.000
Negative Discount              0.000
Too high Discount              0.000
Negative Profit (count)     1901.000
Negative Profit (share)        0.186
dtype: float64
```

18.6% of transactions generate negative profit — requires further category and discount-level investigation.

Categorical Validation

In [290…

```
categorical_cols=['Segment','Region','Category','Ship Mode']
```

In [292…

```
for col in categorical_cols:
    print(f"\n{col}")
    print(orders[col].value_counts(dropna=False))
```

```
Segment
Segment
Consumer       5281
Corporate      3090
Home Office    1823
Name: count, dtype: int64

Region
Region
West       3253
East       2986
Central    2335
South      1620
Name: count, dtype: int64

Category
Category
Office Supplies    6128
Furniture          2201
Technology         1865
Name: count, dtype: int64

Ship Mode
Ship Mode
Standard Class    6120
Second Class      1979
First Class       1548
Same Day           547
Name: count, dtype: int64
```

In [332…

```
missing_values = orders[categorical_cols].isna().sum()
```

In [328…

```
for col,val in missing_values.items():
    print(f'{col}: {val}')
```

```
Segment: 0
Region: 0
Category: 0
Ship Mode: 0
```

No missing values found

Shipping Validation

Invalid Shipping Dates

In [220…

```
orders['ship_days']=(orders['Ship Date']-orders['Order Date']).dt.d
negative_shipping=orders[orders["ship_days"] < 0]
negative_shipping_count=len(negative_shipping)
negative_shipping_count
```

Out[220…

```
0
```

Extreme Shipping Duration

In [223…
```
extreme_shipping = orders[orders["ship_days"] > 30]
extreme_shipping_count=len(extreme_shipping)
extreme_shipping_count
```

Out[223… 0

Logical Validation Summary

In [283…
```
logical_validation_summary = pd.Series({
    "Ship Date < Order Date": negative_shipping_count,
    "Shipping > 30 days": extreme_shipping_count,
})

logical_validation_summary
```

Out[283…
```
Ship Date < Order Date    0
Shipping > 30 days        0
dtype: int64
```

Outlier Review

Top-10 Longest Shipping Duration

In [257…
```
top_ship_days=(orders.sort_values('ship_days',ascending=True).tail(
                [["Order ID", "Customer Name", "Region", "ship_days
top_ship_days
```

Out[257…

	Order ID	Customer Name	Region	ship_days
9409	US-2025-157483	Emily Phan	Central	7
9416	US-2025-143455	Max Ludwig	East	7
2287	US-2023-138219	Bart Pistole	West	7
2300	US-2023-132948	Mark Van Huff	West	7
4567	US-2024-134789	Christine Kargatis	South	7
2324	US-2023-159513	John Dryer	West	7
9393	US-2025-128447	Michael Chen	West	7
8006	US-2025-137085	Carol Triggs	West	7
1699	CA-2022-153623	James Peterman	East	11
1698	CA-2022-153623	James Peterman	East	11

In [360…
```
max_ship_days = orders['ship_days'].max()
count_max_ship_days = (orders['ship_days']==max_ship_days).sum()
count_max_ship_days
```

Out[360… 2

Top-10 Highest Discount

In [347…
```
top_discount=(orders.sort_values('Discount', ascending=False).head(
                                  [["Order ID", "Customer Name", "Cat
                                    "Sales", "Profit", "Discount"]])
top_discount
```

Out[347…

	Order ID	Customer Name	Category	Sales	Profit	Discount
7005	US-2025-118773	Tom Prescott	Office Supplies	12.992	-32.4800	0.8
5763	US-2024-156097	Eugene Hildebrand	Office Supplies	2.308	-3.4620	0.8
5879	US-2024-108644	Sarah Jordon	Office Supplies	1.964	-3.2406	0.8
2525	US-2023-129217	Dennis Pardue	Office Supplies	70.970	-191.6190	0.8
5851	US-2024-132017	Matt Hagelstein	Office Supplies	6.816	-11.5872	0.8
5850	US-2024-149790	Shahid Collister	Office Supplies	15.624	-24.9984	0.8
5848	US-2024-143406	Lisa Ryan	Office Supplies	93.032	-251.1864	0.8
9076	US-2025-163160	Troy Staebel	Office Supplies	96.784	-145.1760	0.8
7915	US-2025-131618	Luke Schmidt	Office Supplies	9.360	-16.3800	0.8
2554	US-2023-117184	Odella Nelson	Office Supplies	33.280	-49.9200	0.8

In [354…
```
max_discount = orders['Discount'].max()
count_max_discount = (orders['Discount']==max_discount).sum()
count_max_discount
```

Out[354… 301

Top-10 Most Negative Profit

In [351…
```
top_negative_profit = (orders.sort_values('Profit',ascending=True).
                [["Order ID", "Customer Name", "Category",
                  "Sales", "Profit", "Discount"]])
top_negative_profit
```

Out[351…

	Order ID	Customer Name	Category	Sales	Profit	Discount
6398	US-2024-108196	Cindy Stewart	Technology	4499.985	-6599.9780	0.7
9318	US-2025-168116	Grant Thornton	Technology	7999.980	-3839.9904	0.5
821	US-2022-169019	Luke Foster	Office Supplies	2177.584	-3701.8928	0.8
7441	US-2025-134845	Sharelle Roach	Technology	2549.985	-3399.9800	0.7
9845	US-2025-122714	Henry Goldwyn	Office Supplies	1889.990	-2929.4845	0.8
4037	US-2023-147830	Natalie Fritzler	Technology	1799.994	-2639.9912	0.7
9557	US-2025-131254	Nathan Cano	Office Supplies	1525.188	-2287.7820	0.8
2108	US-2023-116638	Joseph Holt	Furniture	4297.644	-1862.3124	0.4
4570	US-2024-130946	Zuschuss Carroll	Office Supplies	1088.792	-1850.9464	0.8
226	US-2022-145317	Sean Miller	Technology	22638.480	-1811.0784	0.5

Outlier Summary

In [362…

```
outlier_summary = pd.Series({
    "Max shipping days": orders['ship_days'].max(),
    "Max shipping days count": count_max_ship_days,
    "Min profit": orders['Profit'].min(),
    "Max discount": orders['Discount'].max(),
    "Max discount count": count_max_discount,

})

outlier_summary
```

Out[362…

```
Max shipping days            11.000
Max shipping days count       2.000
Min profit                -6599.978
Max discount                  0.800
Max discount count          301.000
dtype: float64
```

Summary. Part 1

In [864…

```
data_quality_summary = pd.Series({
    "Total rows": total_rows,
    "Total missing values": total_missing,
    "Duplicate Row ID": duplicate_row_id,
    "Invalid discount values": invalid_discount,
    "Non-positive quantity": non_positive_quantity,
    "Negative ship_days": negative_ship_days,
    "Extreme ship_days (>30)": extreme_ship_days,
    "Share of negative profit": negative_profit_share
})

data_quality_summary
```

Out[864…

```
Total rows                  10194.0000
Total missing values            0.0000
Duplicate Row ID                0.0000
Invalid discount values         0.0000
Non-positive quantity           0.0000
Negative ship_days              0.0000
Extreme ship_days (>30)         0.0000
Share of negative profit        0.1865
dtype: float64
```

The dataset contains 10,194 rows with no missing values and no duplicate primary keys, confirming structural integrity. All numeric validation checks passed: No invalid discount values No non-positive quantities No negative shipping durations No extreme shipping delays (>30 days)

The 18.65% share of negative-profit transactions requires further investigation to understand the underlying drivers.

Part 2. KPI & Business Overview

Objective

To assess overall business performance and identify key profit drivers and risk areas across regions, segments, and product categories.

Tasks

1. Calculate Global KPI
2. Build KPI Breakdown by different dimensions
3. Identify Performance Leaders
4. Analyze Losses

Global KPI

In [732…

```
total_sales=orders['Sales'].sum()
total_profit=orders['Profit'].sum()
profit_margin=total_profit/total_sales
total_orders=orders['Order ID'].nunique()
order_sales=orders.groupby('Order ID')['Sales'].sum()
avg_order_value=order_sales.mean()
avg_discount=orders['Discount'].mean()
```

In [734…

```
global_kpi = pd.Series({
    "Total Sales": total_sales,
    "Total Profit": total_profit,
    "Profit Margin": profit_margin,
    "Total Orders": total_orders,
    "Average Order Value": avg_order_value,
    "Average Discount": avg_discount,
})

global_kpi.round(3)
```

Out[734…

```
Total Sales            2326534.354
Total Profit            292296.815
Profit Margin                0.126
Total Orders              5111.000
Average Order Value        455.201
Average Discount             0.155
dtype: float64
```

KPI Breakdown by Region, Segment & Category

BY REGION

In [736…

```
kpi_region=orders.groupby('Region').agg({'Sales':'sum','Profit':'su
kpi_region['Profit Margin']=kpi_region['Profit']/kpi_region['Sales'
unique_orders_region=orders.groupby('Region')['Order ID'].nunique()
kpi_region['Total_Orders']=unique_orders_region
kpi_region=kpi_region.round(3)
kpi_region
```

Out[736…

Region	Sales	Profit	Profit Margin	Total_Orders
Central	503170.673	39865.307	0.079	1179
East	691828.168	94883.260	0.137	1475
South	391721.905	46749.430	0.119	822
West	739813.608	110798.817	0.150	1635

BY SEGMENT

In [738…

```
kpi_segment=orders.groupby('Segment').agg({'Sales':'sum','Profit':'
kpi_segment['Profit Margin']=kpi_segment['Profit']/kpi_segment['Sal
unique_orders_segment=orders.groupby('Segment')['Order ID'].nunique
kpi_segment['Total_Orders']=unique_orders_segment
kpi_segment = kpi_segment.round(3)
kpi_segment
```

Out[738…

Segment	Sales	Profit	Profit Margin	Total_Orders
Consumer	1170659.787	136371.446	0.116	2628
Corporate	715806.135	94249.640	0.132	1552
Home Office	440068.432	61675.728	0.140	931

BY CATEGORY

In [740…

```
kpi_category=orders.groupby('Category').agg({'Sales':'sum','Profit'
kpi_category['Profit Margin']=kpi_category['Profit']/kpi_category['
unique_orders_category=orders.groupby('Category')['Order ID'].nuniq
kpi_category['Total_Orders']=unique_orders_category
kpi_category = kpi_category.round(3)
kpi_category
```

Out[740…

Category	Sales	Profit	Profit Margin	Total_Orders
Furniture	754747.761	19729.996	0.026	1819
Office Supplies	731893.314	126023.443	0.172	3812
Technology	839893.279	146543.376	0.174	1560

Leading Region, Segment & Category

In [742…

```
sales_max_r=kpi_region['Sales'].max()
region_name = kpi_region['Sales'].idxmax()
print(f'Region with the highest sales: {region_name} ({sales_max_r}
```

```
Region with the highest sales: West (739813.608)
```

In [744…
```
margin_max_c=kpi_category['Profit Margin'].max()
category_name = kpi_category['Profit Margin'].idxmax()
print(f'Category with the highest margin: {category_name} ({margin_
```

```
Category with the highest margin: Technology (0.174)
```

In [746…
```
demand_max_s=kpi_segment['Total_Orders'].max()
segment_name = kpi_segment['Total_Orders'].idxmax()
print(f'Segment with the highest demand: {segment_name} ({demand_ma
```

```
Segment with the highest demand: Consumer (2628)
```

Loss Analysis

In [748…
```
orders['is_loss']=orders['Profit']<0
loss_by_cat_share=orders.groupby('Category')['is_loss'].mean()
loss_by_region_share=orders.groupby('Region')['is_loss'].mean()
loss_by_segment_share=orders.groupby('Segment')['is_loss'].mean()
```

In [750…
```
loss_by_cat_share
```

Out[750…
```
Category
Furniture          0.332576
Office Supplies    0.146540
Technology         0.145308
Name: is_loss, dtype: float64
```

In [752…
```
loss_by_region_share
```

Out[752…
```
Region
Central    0.319914
East       0.192230
South      0.159877
West       0.098678
Name: is_loss, dtype: float64
```

In [754…
```
loss_by_segment_share
```

Out[754…
```
Segment
Consumer       0.192767
Corporate      0.181877
Home Office    0.176083
Name: is_loss, dtype: float64
```

In [756…
```
loss_by_cat_sum=orders[orders['Profit'] < 0].groupby('Category')['P
loss_by_region_sum=orders[orders['Profit'] < 0].groupby('Region')['
loss_by_segment_sum=orders[orders['Profit'] < 0].groupby('Segment')
```

In [758…
```
loss_by_cat_sum
```

Out[758…
```
Category
Furniture         -61675.1837
Office Supplies   -56783.8283
Technology        -38579.9182
Name: Profit, dtype: float64
```

In [760…
```
loss_by_region_sum
```

Out[760…
```
Region
Central   -56371.1035
East      -50187.8375
South     -27504.8323
West      -22975.1569
Name: Profit, dtype: float64
```

In [762…
```
loss_by_segment_sum
```

Out[762…
```
Segment
Consumer      -85301.0749
Corporate     -44934.3022
Home Office   -26803.5531
Name: Profit, dtype: float64
```

Loss Summary

In [764…
```
loss_summary_category = pd.concat(
    [loss_by_cat_sum, loss_by_cat_share],
    axis=1
)

loss_summary_category.columns = ['Total_Loss', 'Loss_Share']
loss_summary_category
```

Out[764…

Category	Total_Loss	Loss_Share
Furniture	-61675.1837	0.332576
Office Supplies	-56783.8283	0.146540
Technology	-38579.9182	0.145308

In [766…
```
loss_summary_region = pd.concat(
    [loss_by_region_sum, loss_by_region_share],
    axis=1
)

loss_summary_region.columns = ['Total_Loss', 'Loss_Share']
loss_summary_region
```

Out[766…

Region	Total_Loss	Loss_Share
Central	-56371.1035	0.319914
East	-50187.8375	0.192230
South	-27504.8323	0.159877
West	-22975.1569	0.098678

In [768…

```
loss_summary_segment = pd.concat(
    [loss_by_segment_sum, loss_by_segment_share],
    axis=1
)

loss_summary_segment.columns = ['Total_Loss', 'Loss_Share']
loss_summary_segment
```

Out[768…

Segment	Total_Loss	Loss_Share
Consumer	-85301.0749	0.192767
Corporate	-44934.3022	0.181877
Home Office	-26803.5531	0.176083

Summary. Part 2

In [770…

```
summary_region = kpi_region.join(loss_summary_region)
summary_segment = kpi_segment.join(loss_summary_segment)
summary_category = kpi_category.join(loss_summary_category)
```

In [772…

```
summary_region
```

Out[772…

Region	Sales	Profit	Profit Margin	Total_Orders	Total_Loss	Loss_Sl
Central	503170.673	39865.307	0.079	1179	-56371.1035	0.319
East	691828.168	94883.260	0.137	1475	-50187.8375	0.192
South	391721.905	46749.430	0.119	822	-27504.8323	0.159
West	739813.608	110798.817	0.150	1635	-22975.1569	0.098

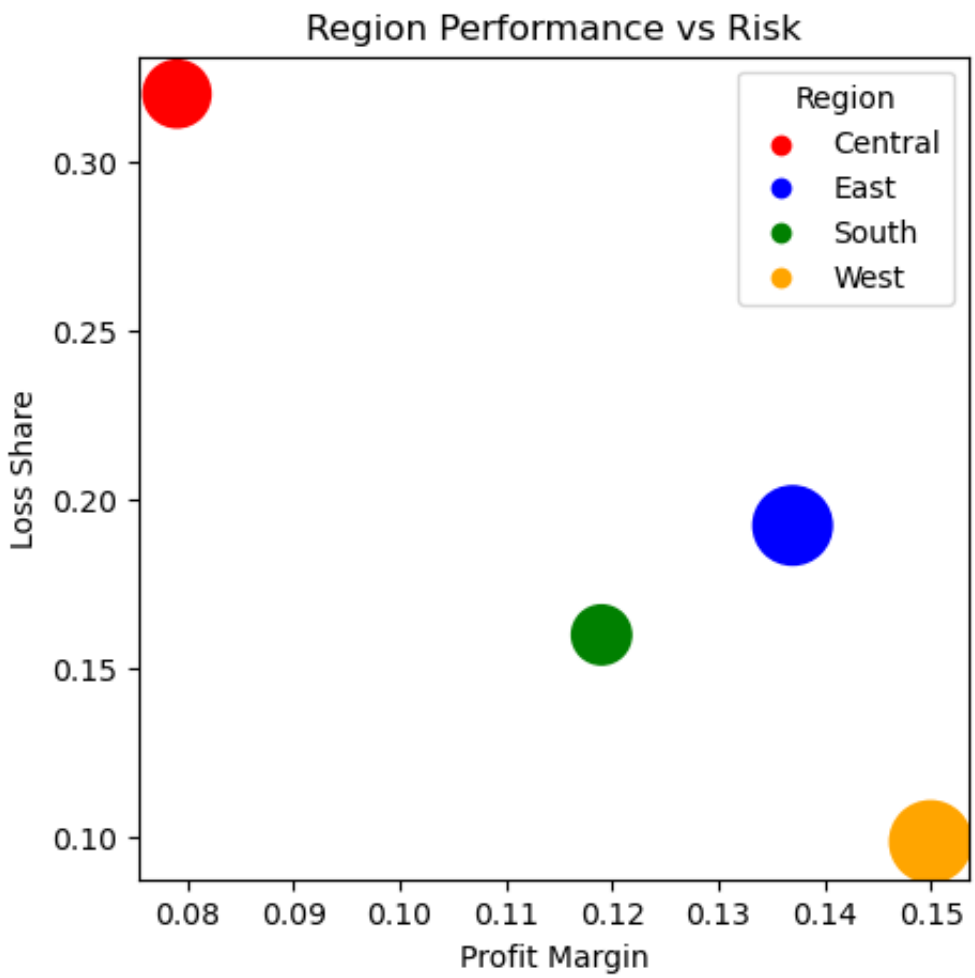
In [870…

```
viz_region = summary_region.plot.scatter(
    x='Profit Margin',
    y='Loss_Share',
    s=summary_region['Sales'] / 1000,
    c=['red', 'blue', 'green','orange'],
    figsize=(5,5)
)

viz_region.set_title('Region Performance vs Risk')
viz_region.set_xlabel('Profit Margin')
viz_region.set_ylabel('Loss Share')

viz_region.scatter([], [], color='red', label='Central')
viz_region.scatter([], [], color='blue', label='East')
viz_region.scatter([], [], color='green', label='South')
```

```
viz_region.scatter([], [], color='orange', label='West')

viz_region.legend(title='Region')
```

Out[870…

```
<matplotlib.legend.Legend at 0x158d25ed0>
```



In [780…

```
summary_segment
```

Out[780…

Segment	Sales	Profit	Profit Margin	Total_Orders	Total_Loss	Los
Consumer	1170659.787	136371.446	0.116	2628	-85301.0749	(
Corporate	715806.135	94249.640	0.132	1552	-44934.3022	
Home Office	440068.432	61675.728	0.140	931	-26803.5531	(

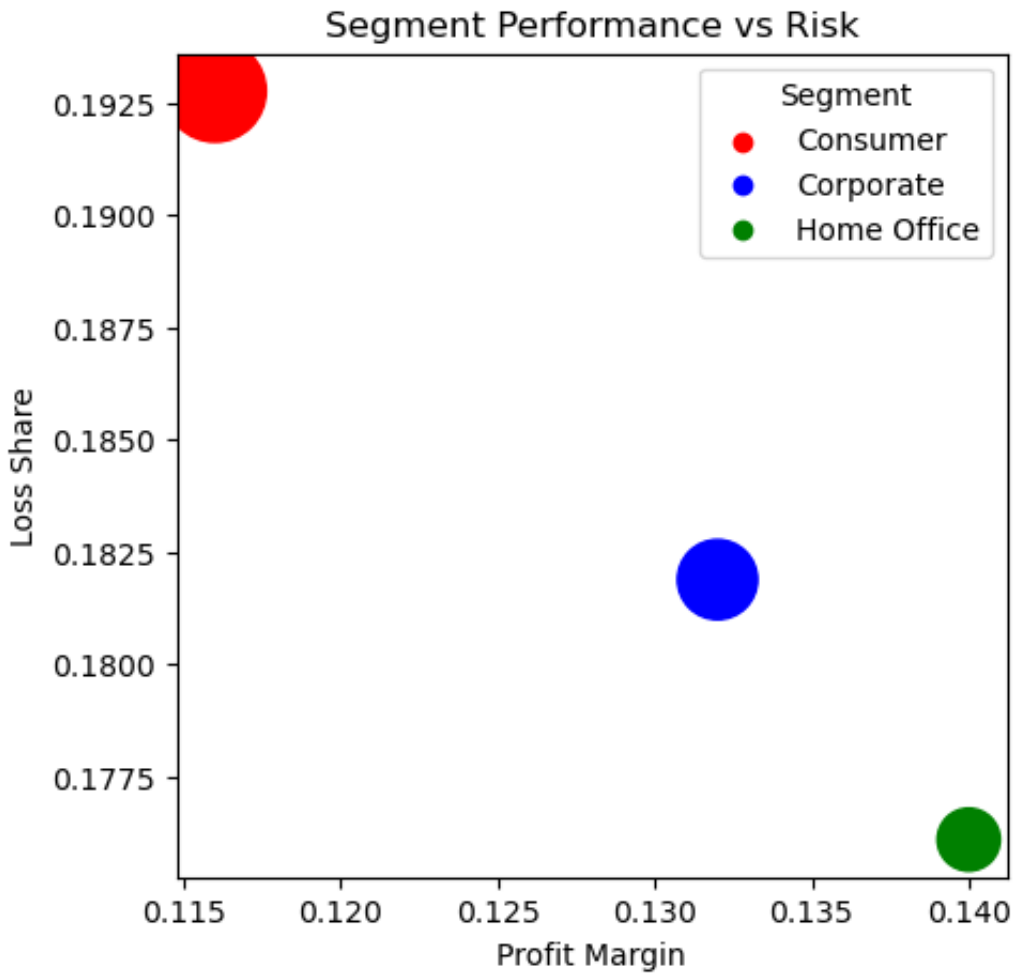
In [880…

```
viz_segment = summary_segment.plot.scatter(
    x='Profit Margin',
    y='Loss_Share',
    s=summary_segment['Sales'] / 1000,
    c=['red', 'blue', 'green',],
    figsize=(5,5)
)
```

```
viz_segment.set_title('Segment Performance vs Risk')
viz_segment.set_xlabel('Profit Margin')
viz_segment.set_ylabel('Loss Share')

viz_segment.scatter([], [], color='red', label='Consumer')
viz_segment.scatter([], [], color='blue', label='Corporate')
viz_segment.scatter([], [], color='green', label='Home Office')

viz_segment.legend(title='Segment')
```

Out[880…

```
<matplotlib.legend.Legend at 0x15a19b990>
```



In [778…

```
summary_category
```

Out[778…

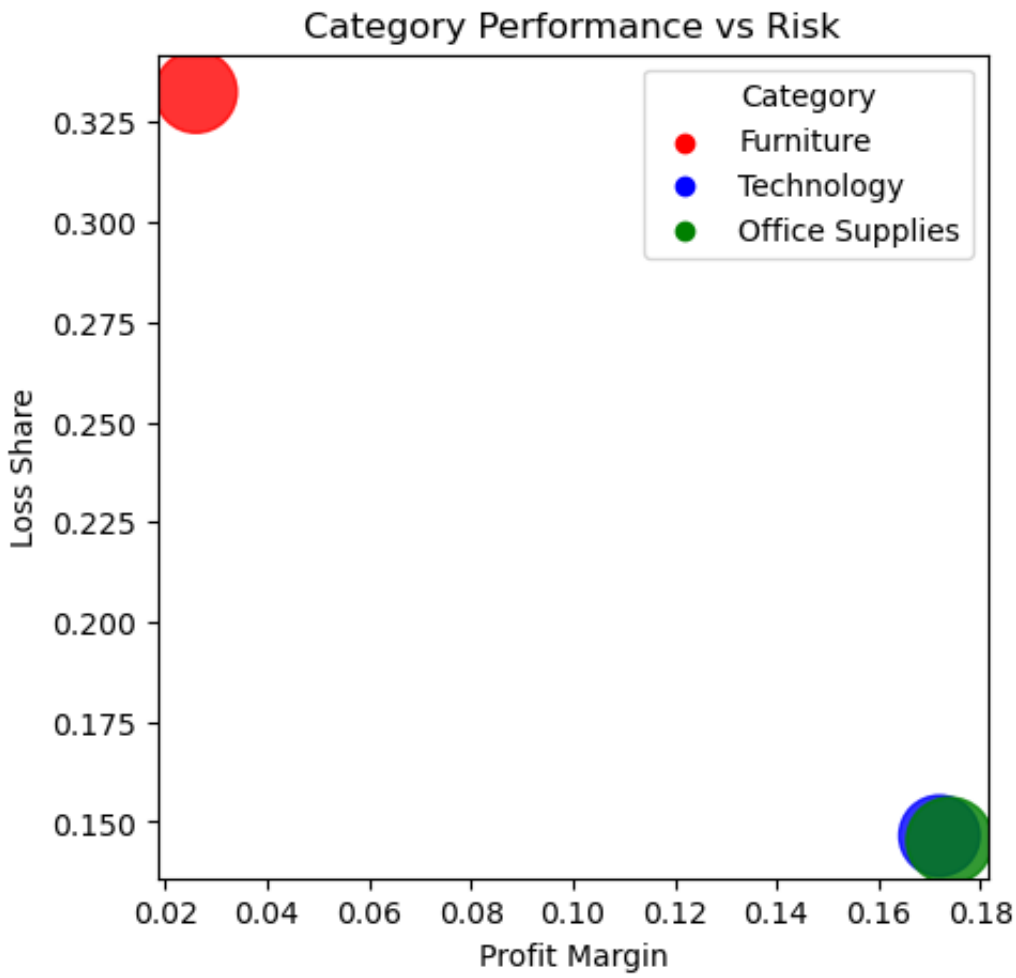
Category	Sales	Profit	Profit Margin	Total_Orders	Total_Loss	Lo
Furniture	754747.761	19729.996	0.026	1819	-61675.1837	
Office Supplies	731893.314	126023.443	0.172	3812	-56783.8283	
Technology	839893.279	146543.376	0.174	1560	-38579.9182	

In [872…

```
viz_category = summary_category.plot.scatter(
    x='Profit Margin',
    y='Loss_Share',
    s=summary_category['Sales'] / 1000,
    c=['red', 'blue', 'green',],
    figsize=(5,5),
    alpha=0.8
)

viz_category.set_title('Category Performance vs Risk')
viz_category.set_xlabel('Profit Margin')
viz_category.set_ylabel('Loss Share')

viz_category.scatter([], [], color='red', label='Furniture')
viz_category.scatter([], [], color='blue', label='Technology')
viz_category.scatter([], [], color='green', label='Office Supplies'

viz_category.legend(title='Category')
```

Out[872…

```
<matplotlib.legend.Legend at 0x158dc5ed0>
```



The business generates $2.33M in sales and $292K in profit, resulting in an actual profit margin of approximately 12.6%. There are 5,111 total orders with an average order value of $455 and an average discount of 15.5%. - Regional Structure: West is the leader in Sales ($739K), profit ($110K), and margin (15%). East is the second most efficient region (13.7% margin). Central has the weakest margin (7.9%) and tje highest share of loss-making transactions (32%). - Segment Structure: Consumer has the highest demand (2,628 orders), Profit ($136K) and Sales ($1.17M). However it shows the highest loss share (~19%), yet the differences between segments are not substantial. - Categorical Structure: Technology and Office Supllies are the most profitable categories (17.4% and 17.2% margin accordingly).. Furniture has a critically low margin (2.6%), 33% loss-making transactions and the highest total loss

amount (-$61K). It can be considered as the primary structural risk within the product portfolio. Loss Concentration: The largest financial losses are concentrated in: Category: Furniture Segment: Consumer Region: Central

Part 3. Discount Impact & Profitability Analysis

Objective

Understand how discount levels affect profitability and identify the discount threshold where margins turn negative.

Tasks

1. Create discount buckets to segment transactions by discount level
2. Calculate Total Sales, Total Profit, Profit Margin, and percentage of negative transcactions among the buckets
3. Find the correlation between Discount and Profit

Discount Buckets

In [969…

```
orders['Bucket']=pd.cut(orders['Discount'], bins=[-0.00001,0,0.2,0.
    labels=['0', '(0–0.2]', '(0.2–0.5]', '(0.5–0.8]'],
    right=True)
orders[['Order ID','Discount','Bucket']]
```

Out[969…

	Order ID	Discount	Bucket
0	US-2022-103800	0.2	(0–0.2]
1	US-2022-112326	0.8	(0.5–0.8]
2	US-2022-112326	0.2	(0–0.2]
3	US-2022-112326	0.2	(0–0.2]
4	US-2022-141817	0.2	(0–0.2]
...	...	...	...
10189	US-2025-143259	0.2	(0–0.2]
10190	US-2025-115427	0.2	(0–0.2]
10191	US-2025-156720	0.2	(0–0.2]
10192	US-2025-143259	0.0	0
10193	CA-2025-143500	0.2	(0–0.2]

10194 rows × 3 columns

Profitability KPI's by Discount Buckets

In [102…
```
bucket_pivot=pd.pivot_table(
    orders,
    index='Bucket',
    values=['Sales', 'Profit', 'is_loss'],
    aggfunc={
        'Sales': 'sum',
        'Profit': 'sum',
        'is_loss': 'mean'
    })
bucket_pivot = bucket_pivot.rename(columns={'is_loss': 'pct_negativ
bucket_pivot['pct_negative']=(bucket_pivot['pct_negative']*100)
```

In [102…
```
bucket_pivot['Margin']=bucket_pivot['Profit']/bucket_pivot['Sales']
```

In [103…
```
bucket_pivot.round(2)
```

Out[103…

Bucket	Profit	Sales	pct_negative	Margin
0	326718.59	1105323.79	0.00	0.30
(0–0.2]	101598.91	856450.41	13.78	0.12
(0.2–0.5]	-58990.50	299868.80	91.68	-0.20
(0.5–0.8]	-77030.19	64891.35	100.00	-1.19

Discount vs Profit Correlation

In [107…
```
correlation=orders['Discount'].corr(orders['Profit']).round(2)
print(f'According to Pearson linear correlation it is equal to {cor
```

```
According to Pearson linear correlation it is equal to -0.22
```

Summary. Part 3

Discount = 0 delivers the strongest profitability (~30% margin) with 0% loss-making transactions. Low discounts (0–0.2) remain profitable (~12% margin) but already introduce losses (~14% of transactions). Discounts above 0.2 are associated with a sharp structural break: (0.2–0.5): -20% margin, ~92% loss-making transactions (0.5–0.8): -119% margin, 100% loss-making transactions Despite the overall correlation being only -0.22, the bucket analysis shows a clear profitability threshold around 20% discount, after which the sales model becomes systematically unprofitable.

Part 4. Sub-Category Profitability Deep Dive

Objective

Identify specific product-level drivers of losses and profitability at the sub-category level.

Tasks

1. Calculate KPI by Sub-Category:
2. Identify: Most profitable sub-categories Most loss-making sub-categories Sub-categories with high loss frequency
3. Compare: Profit contribution vs Sales contribution Margin dispersion across sub-categories
4. Conduct a Pareto (80/20) analysis to evaluate the concentration of total profit across sub-categories.

Sub-Category Overview

In [127…

```
subcat_pivot=pd.pivot_table(
    orders,
    index='Sub-Category',
    values=['Sales', 'Profit', 'is_loss'],
    aggfunc={
        'Sales': 'sum',
        'Profit': 'sum',
        'is_loss': 'mean'
    })
subcat_pivot = subcat_pivot.rename(columns={'is_loss': 'pct_negativ
```

In [127…

```
subcat_pivot['Margin']=subcat_pivot['Profit']/subcat_pivot['Sales']
subcat_pivot['Profit_share']=subcat_pivot['Profit']/orders['Profit'
subcat_pivot['Sales_share']=subcat_pivot['Sales']/orders['Sales'].s
```

In [127…

```
subcat_pivot.round(3)
```

Out[127…

Sub-Category	Profit	Sales	pct_negative	Margin	Profit_share	Sa
Accessories	41936.636	167380.318	0.117	0.251	0.143	
Appliances	18329.484	108213.185	0.141	0.169	0.063	
Art	6653.196	27659.014	0.001	0.241	0.023	
Binders	31426.100	207354.881	0.400	0.152	0.108	
Bookcases	-3632.074	115361.204	0.478	-0.031	-0.012	
Chairs	27223.532	335768.249	0.375	0.081	0.093	
Copiers	56093.936	150745.290	0.000	0.372	0.192	
Envelopes	6988.025	16528.362	0.000	0.423	0.024	
Fasteners	2428.636	8532.240	0.052	0.285	0.008	
Furnishings	13891.743	95598.126	0.176	0.145	0.048	
Labels	5572.778	12695.042	0.005	0.439	0.019	
Machines	3461.977	189925.031	0.376	0.018	0.012	
Paper	34511.507	79540.538	0.001	0.434	0.118	
Phones	45050.826	331842.640	0.151	0.136	0.154	
Storage	21285.111	224644.554	0.190	0.095	0.073	
Supplies	-1171.395	46725.498	0.172	-0.025	-0.004	
Tables	-17753.206	208020.182	0.629	-0.085	-0.061	

Loss-Making Sub-Categories Analysis

In [128…

```
top3_profit_subcat=subcat_pivot.sort_values('Profit', ascending=Fal
top3_profit_subcat
```

Out[128…

Sub-Category	Profit	Sales	pct_negative	Margin	Profit_share	Sales
Copiers	56093.94	150745.29	0.00	0.37	0.19	
Phones	45050.83	331842.64	0.15	0.14	0.15	
Accessories	41936.64	167380.32	0.12	0.25	0.14	

In [128…

```
top3_loss_subcat=subcat_pivot.sort_values('Profit', ascending=True)
top3_loss_subcat
```

Out[128…

Sub-Category	Profit	Sales	pct_negative	Margin	Profit_share	Sales_
Tables	-17753.21	208020.18	0.63	-0.09	-0.06	
Bookcases	-3632.07	115361.20	0.48	-0.03	-0.01	
Supplies	-1171.39	46725.50	0.17	-0.03	-0.00	

In [128…

```
top3_losspct_subcat=subcat_pivot.sort_values('pct_negative', ascend
top3_losspct_subcat
```

Out[128…

Sub-Category	Profit	Sales	pct_negative	Margin	Profit_share	Sales_
Tables	-17753.21	208020.18	0.63	-0.09	-0.06	
Bookcases	-3632.07	115361.20	0.48	-0.03	-0.01	
Binders	31426.10	207354.88	0.40	0.15	0.11	

Efficiency Analysis

In [129…

```
efficient_subcat=subcat_pivot[subcat_pivot['Profit_share']>subcat_p
efficient_subcat
```

Out[129…

Sub-Category	Profit	Sales	pct_negative	Margin	Profit_share
Accessories	41936.6357	167380.318	0.117419	0.250547	0.143473
Appliances	18329.4844	108213.185	0.141350	0.169383	0.062708
Art	6653.1962	27659.014	0.001218	0.240544	0.022762
Binders	31426.1003	207354.881	0.399871	0.151557	0.107514
Copiers	56093.9365	150745.290	0.000000	0.372111	0.191907
Envelopes	6988.0247	16528.362	0.000000	0.422790	0.023907
Fasteners	2428.6358	8532.240	0.052402	0.284642	0.008309
Furnishings	13891.7430	95598.126	0.176412	0.145314	0.047526
Labels	5572.7780	12695.042	0.005435	0.438973	0.019065
Paper	34511.5070	79540.538	0.000723	0.433886	0.118070
Phones	45050.8265	331842.640	0.150609	0.135760	0.154127

In [129…

```
not_efficient_subcat=subcat_pivot[subcat_pivot['Profit_share']<subc
not_efficient_subcat
```

Out[129…

Sub-Category	Profit	Sales	pct_negative	Margin	Profit_share
Bookcases	-3632.0736	115361.2043	0.478448	-0.031484	-0.012426
Chairs	27223.5323	335768.2490	0.375394	0.081078	0.093137
Machines	3461.9769	189925.0310	0.376068	0.018228	0.011844
Storage	21285.1115	224644.5540	0.190421	0.094750	0.072820
Supplies	-1171.3945	46725.4980	0.171875	-0.025070	-0.004008
Tables	-17753.2061	208020.1820	0.628834	-0.085344	-0.060737

In [129…

```
margin_std_subcat=subcat_pivot['Margin'].std()
margin_std_subcat
```

Out[129… 0.1679618621924883

In [129…

```
margindiff_subcat=subcat_pivot['Margin'].max() - subcat_pivot['Marg
```

Pareto Analysis

In [130…

```
subcat_pivot=subcat_pivot.sort_values('Profit', ascending=False)
subcat_pivot['Profit_share_subcat']=(subcat_pivot['Profit'] / subca
subcat_pivot['Cumulative_profit_%'] = (subcat_pivot['Profit_share']
subcat_pivot
```

Out[130…

Sub-Category	Profit	Sales	pct_negative	Margin	Profit_shar
Copiers	56093.9365	150745.2900	0.000000	0.372111	0.19190
Phones	45050.8265	331842.6400	0.150609	0.135760	0.15412
Accessories	41936.6357	167380.3180	0.117419	0.250547	0.14347
Paper	34511.5070	79540.5380	0.000723	0.433886	0.11807
Binders	31426.1003	207354.8810	0.399871	0.151557	0.10751
Chairs	27223.5323	335768.2490	0.375394	0.081078	0.09313
Storage	21285.1115	224644.5540	0.190421	0.094750	0.07282
Appliances	18329.4844	108213.1850	0.141350	0.169383	0.06270
Furnishings	13891.7430	95598.1260	0.176412	0.145314	0.04752
Envelopes	6988.0247	16528.3620	0.000000	0.422790	0.02390
Art	6653.1962	27659.0140	0.001218	0.240544	0.02276
Labels	5572.7780	12695.0420	0.005435	0.438973	0.01906
Machines	3461.9769	189925.0310	0.376068	0.018228	0.01184
Fasteners	2428.6358	8532.2400	0.052402	0.284642	0.00830
Supplies	-1171.3945	46725.4980	0.171875	-0.025070	-0.00400
Bookcases	-3632.0736	115361.2043	0.478448	-0.031484	-0.01242
Tables	-17753.2061	208020.1820	0.628834	-0.085344	-0.06073

In [130…

```
top80_pct=subcat_pivot[subcat_pivot['Cumulative_profit_%']<=85]
top80_pct
```

Out[130…

Sub-Category	Profit	Sales	pct_negative	Margin	Profit_share
Copiers	56093.9365	150745.290	0.000000	0.372111	0.191907
Phones	45050.8265	331842.640	0.150609	0.135760	0.154127
Accessories	41936.6357	167380.318	0.117419	0.250547	0.143473
Paper	34511.5070	79540.538	0.000723	0.433886	0.118070
Binders	31426.1003	207354.881	0.399871	0.151557	0.107514
Chairs	27223.5323	335768.249	0.375394	0.081078	0.093137

Summary. Part 4

Most profitable sub-categories are Copiers, Phones, and Accessories, characterized by strong absolute profit, positive margins, and relatively low loss frequency. The most loss-making sub-categories are Tables, Bookcases, and Supplies, with negative margins (-8.5%, -3.1%, and -2.5% accordingly) and high percentages of loss-making transactions (63%, 48%, and 17%). Binders can also be considered structurally risky due to a high share of negative transactions (40%). However, its overall contribution remains positive, as its profit share exceeds its sales share. According to the Pareto analysis, Copiers, Phones, Accessories, Paper, Binders, and Chairs generate approximately 81% of total profit, indicating strong profit concentration within a limited number of sub-categories. The wide margin range, combined with a relatively high standard deviation, indicates significant margin dispersion. This suggests structural imbalances within the product portfolio. Such a product mix may negatively impact overall business performance.

Part 5. Customer Analytics

Objective

Customer Segmentation

Tasks

1. Calculate Customer-level KPIs
2. Find Top-10 & loss-making customers
3. Create RFM Sefmentation
4. Analyze Segment vs Profitability relationship

Customer-level KPIs

In [137…

```
customer_pivot=pd.pivot_table(
    orders,
    index='Customer Name',
    values=['Sales', 'Profit','Discount','Order ID'],
    aggfunc={
```

```
        'Sales': 'sum',
        'Profit': 'sum',
        'Discount': 'mean',
        'Order ID': 'nunique'})
customer_pivot = customer_pivot.rename(
    columns={'Order ID': 'Orders Count'}
)
customer_pivot = customer_pivot[['Sales', 'Profit', 'Orders Count',
customer_pivot.round(3)
```

Out[137…

Customer Name	Sales	Profit	Orders Count	Discount
Aaron Bergman	886.156	129.346	3	0.067
Aaron Hawkins	1744.700	365.215	7	0.091
Aaron Smayling	3050.692	-253.575	7	0.355
Adam Bellavance	7755.620	2054.588	8	0.044
Adam Hart	3250.337	281.189	10	0.135
...	...	...	...	...
Xylona Preis	2503.156	675.958	13	0.044
Yana Sorensen	6720.444	1778.292	8	0.050
Yoseph Carroll	5454.350	1305.629	5	0.075
Zuschuss Carroll	8025.707	-1032.149	13	0.255
Zuschuss Donatelli	1493.944	249.131	5	0.111

800 rows × 4 columns

Top-10 loss-making clients

In [138…

```
top10_profit_customers=customer_pivot.sort_values('Profit', ascendi
top10_profit_customers
```

Out[138…

Customer Name	Sales	Profit	Orders Count	Discount
Tamara Chand	19052.22	8981.32	5	0.12
Raymond Buch	15117.34	6976.10	6	0.09
Sanjit Chand	14142.33	5757.41	9	0.06
Hunter Lopez	12873.30	5622.43	6	0.02
Adrian Barton	14473.57	5444.81	10	0.24
Tom Ashbrook	14595.62	4703.79	4	0.08
Christopher Martinez	8954.02	3899.89	4	0.12
Keith Dawkins	8181.26	3038.63	12	0.09
Andy Reiter	6608.45	2884.62	6	0.07
Daniel Raglin	8350.87	2869.08	8	0.15

In [139…

```
neg_profit_customers=customer_pivot[customer_pivot['Profit']<0]
neg_profit_customers=neg_profit_customers.sort_values('Profit', asc
neg_profit_customers
```

Out[139…

Customer Name	Sales	Profit	Orders Count	Discount
Cindy Stewart	5690.06	-6626.39	6	0.20
Grant Thornton	9351.21	-4108.66	3	0.25
Luke Foster	3930.51	-3583.98	7	0.32
Sharelle Roach	3233.48	-3333.91	5	0.37
Henry Goldwyn	3247.64	-2797.96	12	0.17
…	…	…	…	…
Thais Sissman	4.83	-3.32	2	0.70
Adrian Hane	1735.51	-2.31	7	0.24
Mitch Gastineau	16.74	-1.25	1	0.45
Paul Lucas	239.48	-0.75	5	0.25
Jasper Cacioppo	71.26	-0.36	4	0.22

158 rows × 4 columns

In [141…

```
pct_negative_customers = ((customer_pivot['Profit'] < 0).sum()
    / orders['Customer Name'].nunique()) * 100
pct_negative_customers
```

Out[141… 19.75

RFM Segmentation

In [142…
```
reference_date = orders['Order Date'].max()
recency = (
    reference_date -
    orders.groupby('Customer Name')['Order Date'].max()
).dt.days
recency
```

Out[142…
```
Customer Name
Aaron Bergman         415
Aaron Hawkins          12
Aaron Smayling         88
Adam Bellavance        54
Adam Hart              34
                      ...
Xylona Preis           43
Yana Sorensen           9
Yoseph Carroll          4
Zuschuss Carroll       54
Zuschuss Donatelli    202
Name: Order Date, Length: 800, dtype: int64
```

In [142…
```
frequency = orders.groupby('Customer Name')['Order ID'].nunique()
frequency
```

Out[142…
```
Customer Name
Aaron Bergman          3
Aaron Hawkins          7
Aaron Smayling         7
Adam Bellavance        8
Adam Hart             10
                      ..
Xylona Preis          13
Yana Sorensen          8
Yoseph Carroll         5
Zuschuss Carroll      13
Zuschuss Donatelli     5
Name: Order ID, Length: 800, dtype: int64
```

In [142…
```
monetary = orders.groupby('Customer Name')['Profit'].sum()
monetary
```

Out[142…

```
Customer Name
Aaron Bergman          129.3465
Aaron Hawkins          365.2152
Aaron Smayling        -253.5746
Adam Bellavance       2054.5885
Adam Hart              281.1890
                         ...
Xylona Preis           675.9584
Yana Sorensen         1778.2923
Yoseph Carroll        1305.6290
Zuschuss Carroll     -1032.1490
Zuschuss Donatelli     249.1307
Name: Profit, Length: 800, dtype: float64
```

In [144…

```
rfm = pd.concat([recency, frequency, monetary], axis=1)
rfm.columns = ['Recency', 'Frequency', 'Monetary']
rfm
```

Out[144…

Customer Name	Recency	Frequency	Monetary
Aaron Bergman	415	3	129.3465
Aaron Hawkins	12	7	365.2152
Aaron Smayling	88	7	-253.5746
Adam Bellavance	54	8	2054.5885
Adam Hart	34	10	281.1890
...	...	...	...
Xylona Preis	43	13	675.9584
Yana Sorensen	9	8	1778.2923
Yoseph Carroll	4	5	1305.6290
Zuschuss Carroll	54	13	-1032.1490
Zuschuss Donatelli	202	5	249.1307

800 rows × 3 columns

In [146…

```
rfm['R_score'] = pd.qcut(rfm['Recency'],5,labels=[5,4,3,2,1])
rfm['F_score'] = pd.qcut(rfm['Frequency'].rank(method='first'),5,la
rfm['M_score'] = pd.qcut(rfm['Monetary'].rank(method='first'),5,lab
```

In [147…

```
rfm['RFM_score'] = (rfm['R_score'].astype(int) + rfm['F_score'].ast
rfm.sort_values('RFM_score', ascending=False)
```

Out[147…

Customer Name	Recency	Frequency	Monetary	R_score	F_score	M_score	RF
Annie Thurman	13	10	974.1070	5	5	5	
Arianne Irving	13	10	867.7271	5	5	5	
James Galang	1	11	1415.6664	5	5	5	
Harry Marie	2	10	2437.9836	5	5	5	
Chris Selesnick	26	12	738.3626	5	5	5	
...	...	...	...	...	...	...	
Pauline Chand	882	2	-184.3364	1	1	1	
Becky Martin	307	4	-1659.9581	1	1	1	
Harold Dahlen	769	3	-135.8760	1	1	1	
Andy Gerbode	845	4	-152.7613	1	1	1	
Paul Knutson	483	2	-798.7050	1	1	1	

800 rows × 7 columns

In [150…

```
def segment_customer(row):
    if row['RFM_score'] >= 12:
        return 'High Value'
    elif row['RFM_score'] >= 9:
        return 'Loyal'
    elif row['RFM_score'] >= 4:
        return 'At Risk'
    else:
        return 'Low Value '
rfm['Segment'] = rfm.apply(segment_customer, axis=1)
```

In [150…

```
rfm.sort_values('RFM_score', ascending=False)
```

Out[150…

Customer Name	Recency	Frequency	Monetary	R_score	F_score	M_score	RF
Annie Thurman	13	10	974.1070	5	5	5	
Arianne Irving	13	10	867.7271	5	5	5	
James Galang	1	11	1415.6664	5	5	5	
Harry Marie	2	10	2437.9836	5	5	5	
Chris Selesnick	26	12	738.3626	5	5	5	
...	...	...	...	...	...	...	
Pauline Chand	882	2	-184.3364	1	1	1	
Becky Martin	307	4	-1659.9581	1	1	1	
Harold Dahlen	769	3	-135.8760	1	1	1	
Andy Gerbode	845	4	-152.7613	1	1	1	
Paul Knutson	483	2	-798.7050	1	1	1	

800 rows × 8 columns

Profitability by Customer Segment

In [157…

```
segment_profit = rfm.merge(
    customer_pivot[['Sales','Profit']],
    left_index=True,
    right_index=True)

segment_agg = segment_profit.groupby('Segment').agg({
    'Sales': 'sum',
    'Profit': 'sum'})

segment_agg['Margin'] = (segment_agg['Profit'] / segment_agg['Sales

segment_agg['Profit_share_%'] = (segment_agg['Profit'] / segment_ag

segment_agg['Avg_Profit_per_Customer'] = (segment_profit.groupby('S
```

```
segment_agg
```

Out[157…

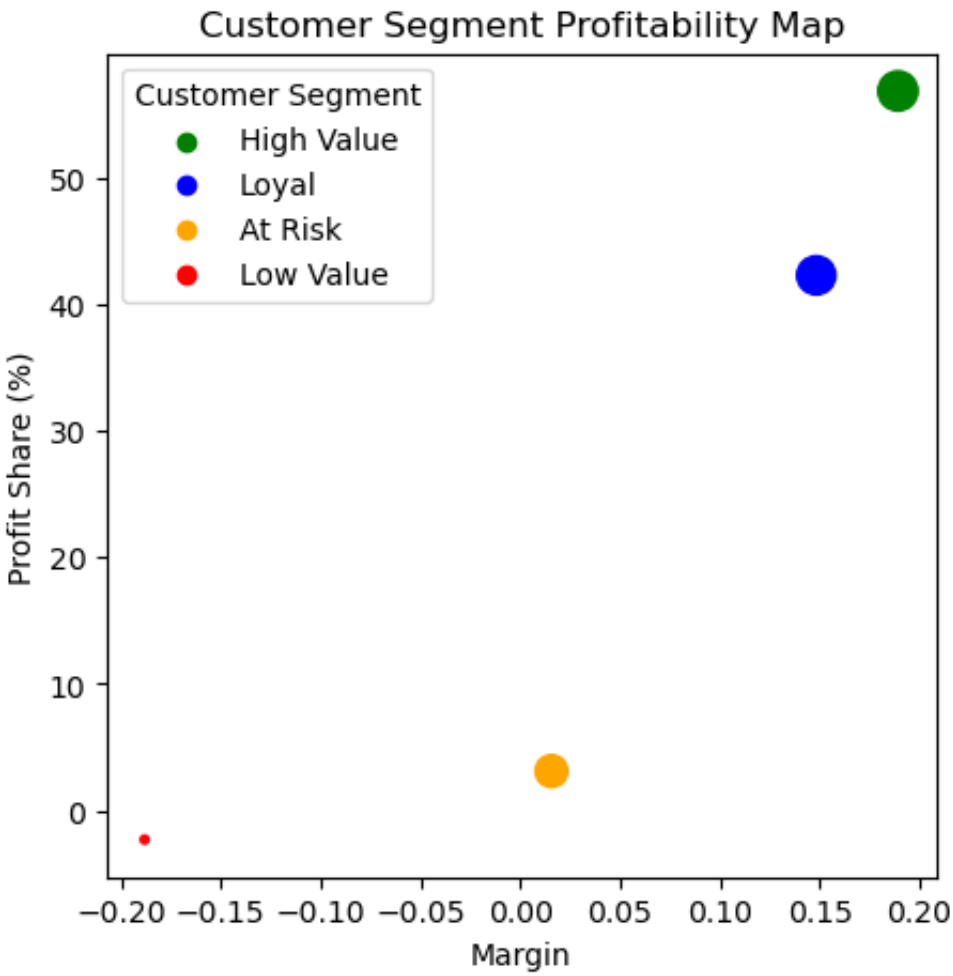
Segment	Sales	Profit	Margin	Profit_share_%	Avg_Profit_
At Risk	578818.4027	9102.7986	0.015727	3.11	
High Value	878437.8795	166284.3778	0.189296	56.89	
Low Value	35885.4335	-6749.9234	-0.188096	-2.31	
Loyal	833392.6386	123659.5616	0.148381	42.31	

In [159…

```
viz_customer_segment = segment_agg.plot.scatter(
    x='Margin',
    y='Profit_share_%',
    s=segment_agg['Sales'] / 5000,
    c=['orange', 'green', 'red', 'blue'],
    figsize=(5,5)
)

viz_customer_segment.set_title('Customer Segment Profitability Map'
viz_customer_segment.set_xlabel('Margin')
viz_customer_segment.set_ylabel('Profit Share (%)')

viz_customer_segment.scatter([], [], color='green', label='High Val
viz_customer_segment.scatter([], [], color='blue', label='Loyal')
viz_customer_segment.scatter([], [], color='orange', label='At Risk
viz_customer_segment.scatter([], [], color='red', label='Low Value'

viz_customer_segment.legend(title='Customer Segment')
```

Out[159…

```
<matplotlib.legend.Legend at 0x14e075ed0>
```



Summary. Part 5

Approximately 19.75% of customers are overall unprofitable. Nearly one in five customers generates negative total profit, indicating structural inefficiencies in pricing, discounting, or customer selection strategy. This highlights that profitability is concentrated in a relatively small portion of the customer base. "High Value" segment generates the majority of total profit (≈57%) with the highest margin (~19%) and the highest average profit per customer. The business is strongly dependent on this group. "Loyal" customers contribute a substantial share of total profit (≈42%)

with solid margins (~15%). This segment represents a stable and sustainable revenue foundation. Although "At Risk" segment generates significant sales volume, its margin is very low (~1.6%) and its contribution to total profit is minimal (~3%). These customers are revenue-heavy but inefficient from a profitability standpoint. "Low Value" segment has negative margins (~-19%) and generates overall losses. It destroys value and requires pricing or engagement strategy reassessment.

Part 6. Time Series & Trend Analysis

Objective

Analyze business performance dynamics over time.

Tasks

1. Build: Monthly Sales, Monthly Profit, Monthly Margin
2. Calculate: Month-over-Month (MoM) growth and Year-over-Year (YoY) growth
3. Identify seasonality patterns.

Monthly Sales, Monthly Profit, Monthly Margin

In [164…
```
orders['Year_Month'] = orders['Order Date'].dt.to_period('M')
```

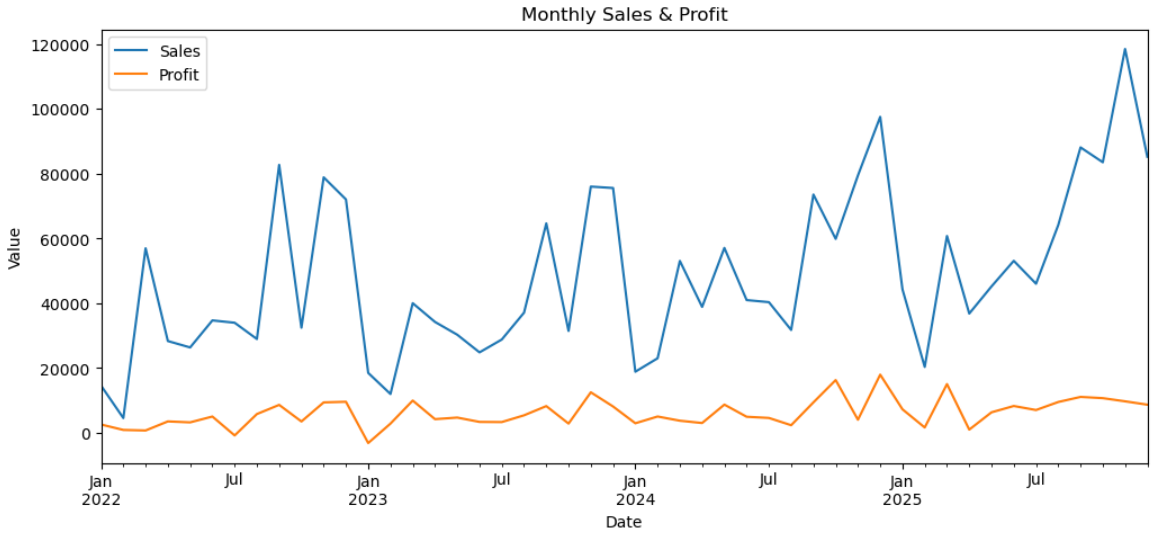
In [165…
```
monthly = orders.groupby('Year_Month').agg({
    'Sales': 'sum',
    'Profit': 'sum'
})

monthly['Margin'] = monthly['Profit'] / monthly['Sales']

monthly.index = monthly.index.to_timestamp()
```

In [173…
```
correlation = monthly['Sales'].corr(monthly['Profit']).round(2)
print(f'The Pearson linear correlation coefficient shows a strong p
```

```
The Pearson linear correlation coefficient shows a strong positive c
orrelation of 0.72
```

In [173…
```
viz_monthly = monthly[['Sales','Profit']].plot(
    figsize=(12,5),
    title='Monthly Sales & Profit')

viz_monthly.set_xlabel('Date')
viz_monthly.set_ylabel('Value')
```

Out[173…
```
Text(0, 0.5, 'Value')
```

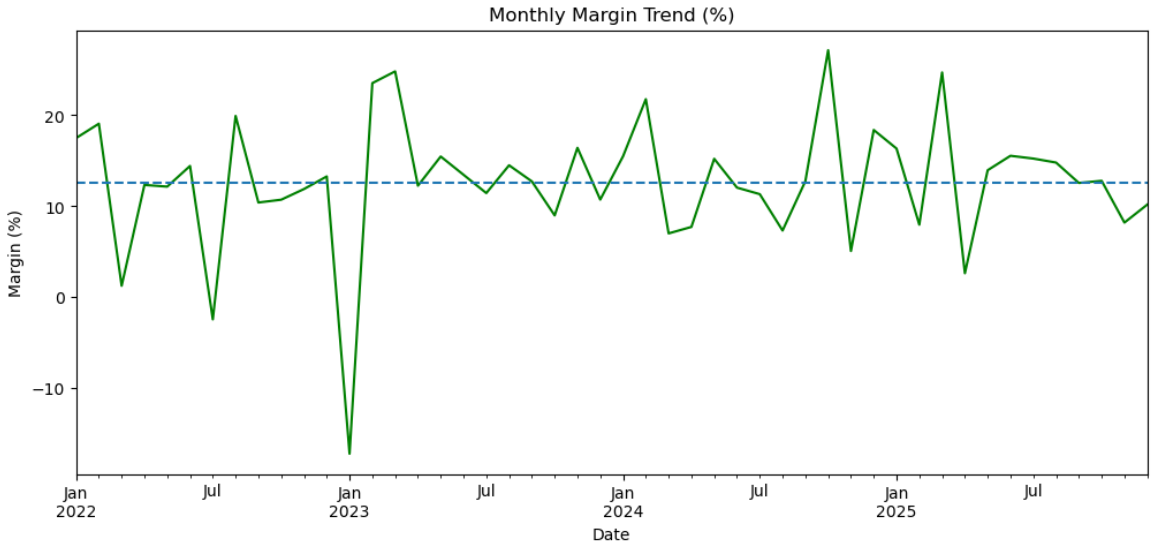


In [165…

```
viz_monthly_margin = (monthly['Margin'] * 100).plot(
    figsize=(12,5),
    title='Monthly Margin Trend (%)',
    color='green'
)

viz_monthly_margin.axhline(
    y=monthly['Margin'].mean() * 100,
    linestyle='--'
)

viz_monthly_margin.set_xlabel('Date')
viz_monthly_margin.set_ylabel('Margin (%)')
```

Out[165…

```
Text(0, 0.5, 'Margin (%)')
```



In [171…

```
margin_mean = monthly['Margin'].mean()
margin_std = monthly['Margin'].std()

anomalies = monthly[monthly['Margin'] < (margin_mean - 2 * margin_s
anomalies
```

Out[171…

Year_Month	Sales	Profit	Margin	MoM_Sales_%	MoM_Profit
2022-07-01	33946.3930	-841.4826	-0.024789	-2.09	-116
2023-01-01	18461.9156	-3189.8030	-0.172777	-74.36	-133

MoM and YoY growth

In [166…

```
monthly['MoM_Sales_%'] = (monthly['Sales'].pct_change() * 100).roun
monthly['MoM_Profit_%'] = (monthly['Profit'].pct_change() * 100).ro
```

In [166…

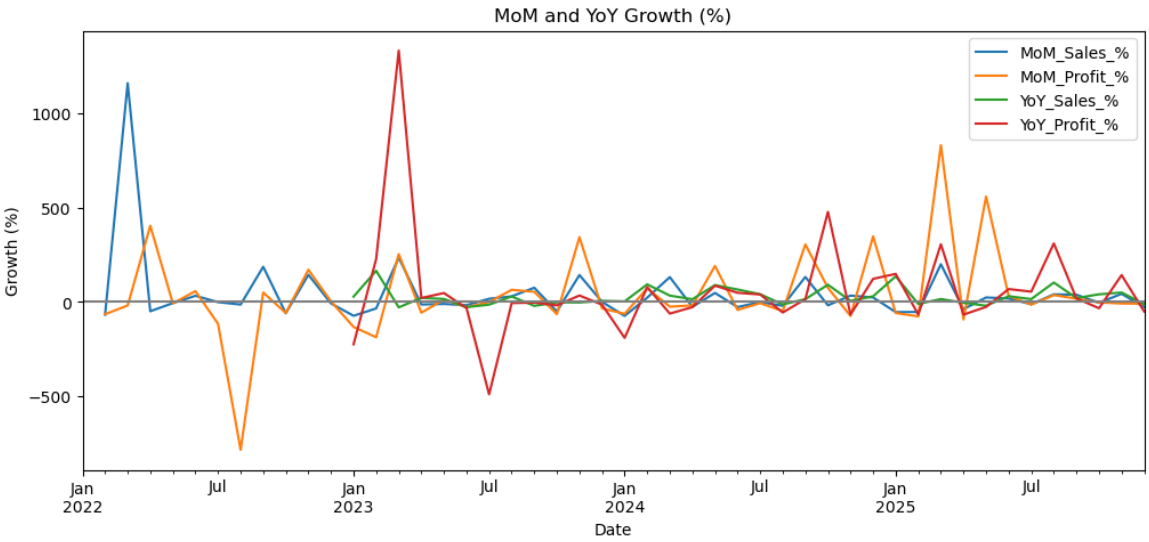
```
monthly['YoY_Sales_%'] = (monthly['Sales'].pct_change(12) * 100).ro
monthly['YoY_Profit_%'] = (monthly['Profit'].pct_change(12) * 100).
```

In [168…

```
viz_monthly_sum = monthly[
    ['MoM_Sales_%',
     'MoM_Profit_%',
     'YoY_Sales_%',
     'YoY_Profit_%']
].plot(
    figsize=(12,5),
    title='MoM and YoY Growth (%)'
)

viz_monthly_sum.set_xlabel('Date')
viz_monthly_sum.set_ylabel('Growth (%)')
viz_monthly_sum.axhline(0, linestyle='-',c='grey')
```

Out[168…

```
<matplotlib.lines.Line2D at 0x14feb3410>
```

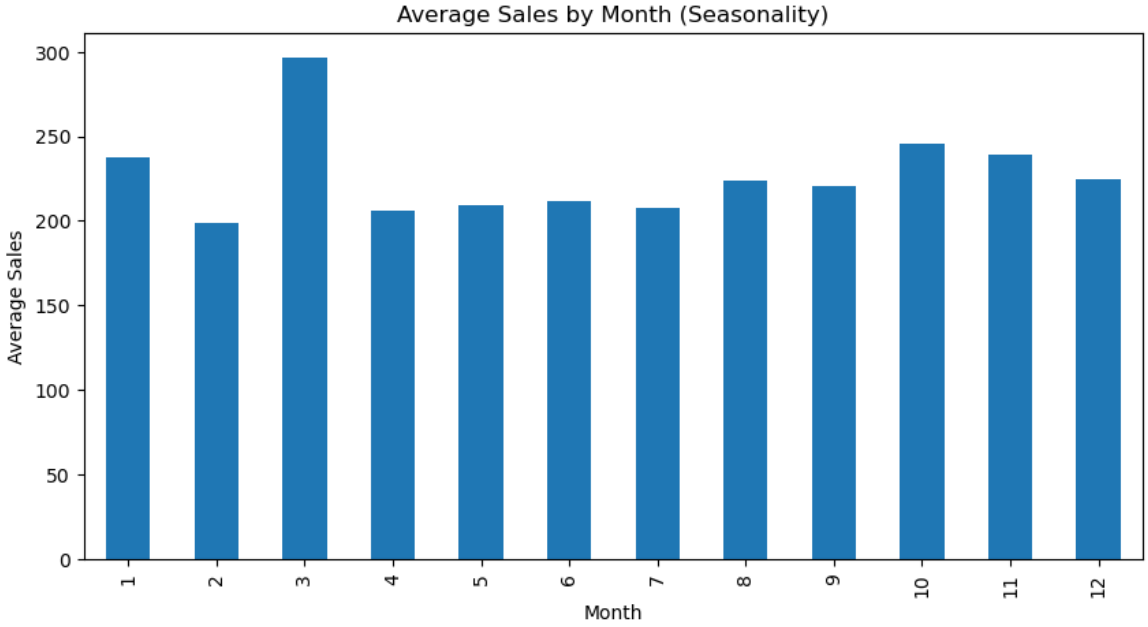


Seasonality patterns

In [168…
```
orders['Month'] = orders['Order Date'].dt.month
seasonality = orders.groupby('Month').agg({'Sales': 'mean','Profit'
seasonality['Margin'] = seasonality['Profit'] / seasonality['Sales'
```

In [168…
```
viz_seasonality = seasonality['Sales'].plot(
    kind='bar',
    figsize=(10,5),
    title='Average Sales by Month (Seasonality)'
)

viz_seasonality.set_xlabel('Month')
viz_seasonality.set_ylabel('Average Sales')
```

Out[168…
```
Text(0, 0.5, 'Average Sales')
```



Summary. Part 6

Sales demonstrate a clear upward trajectory over time. Profit follows sales growth but with much volatility. The Pearson linear correlation coefficient shows a strong positive correlation between Sales and Profit of 0.72. Average Margin remains positive at the level of 12-13$. The analysis detected two statistically significant anomaly months: July 2022, January 2023. Seasonality shows that March usually has the highest average sales.

Part 7. Shipping & Operational Efficiency

Objective

Assess the impact of logistics on profitability.

Tasks

1. Calculate average shipping days by region and shipping mode

2. Analyze shipping days vs profit and return rate
3. Identify the most expensive and the most profitable shipping method
4. Determine whether faster delivery impacts profitability.

Average shipping days by region and shipping mode

In [178…

```
avg_ship = (
    orders
        .groupby(['Region', 'Ship Mode'])['ship_days']
        .mean()
        .to_frame('avg_ship_days')
        .groupby(level=0, group_keys=False)
        .apply(lambda x: x.sort_values('avg_ship_days', ascending=T
).round(2)

avg_ship
```

Out[178…

Region	Ship Mode	avg_ship_days
Central	Same Day	0.03
	First Class	2.31
	Second Class	3.33
	Standard Class	4.98
East	Same Day	0.00
	First Class	2.23
	Second Class	3.24
	Standard Class	4.94
South	Same Day	0.01
	First Class	2.11
	Second Class	3.19
	Standard Class	5.00
West	Same Day	0.10
	First Class	2.10
	Second Class	3.19
	Standard Class	5.05

Shipping days vs profit and return rate

In [193…
```
ship_profit = (orders.groupby('ship_days')['Profit'].mean()).to_fra
ship_profit
```

Out[193…

ship_days	Avg_Profit
0	29.933
1	20.437
2	39.580
3	26.522
4	25.274
5	27.876
6	27.661
7	32.488
11	35.414

In [193…
```
orders = (orders.merge(returns[['Order ID']],on='Order ID',how='lef
orders['is_returned'] = (orders['_merge'] == 'both').astype(int)
orders.drop(columns=['_merge'], inplace=True)
```

In [194…
```
return_by_days = (orders.groupby('ship_days')['returned_flag'].mean
return_by_days
```

Out[194…

ship_days	Return Rate
0	0.122
1	0.092
2	0.074
3	0.096
4	0.071
5	0.062
6	0.088
7	0.092
11	0.000

The most expensive and the most profitable shipping method

In [192…
```
sales_by_shipmode = (orders.groupby('Ship Mode')['Sales'].mean()).r
sales_by_shipmode
```

Out[192…

Ship Mode	Avg Sales
First Class	227.229
Same Day	236.329
Second Class	235.812
Standard Class	225.301

In [192…
```
profit_by_shipmode = (orders.groupby('Ship Mode')['Profit'].mean())
profit_by_shipmode
```

Out[192…

Ship Mode	Avg Profit
First Class	31.662
Same Day	29.544
Second Class	29.794
Standard Class	27.477

Faster delivery vs profitability

In [191…
```
orders['ship_days'].corr(orders['Profit']).round(3)
```

Out[191…
```
-0.004
```

In [191…
```
profit_by_days = (orders.groupby('ship_days')['Profit'].mean().to_f

profit_by_days
```

Out[191…

ship_days	Avg_Profit
0	29.933
1	20.437
2	39.580
3	26.522
4	25.274
5	27.876
6	27.661
7	32.488
11	35.414

In [191…

```
median_days = orders['ship_days'].median()

orders['speed_group'] = orders['ship_days'].apply(lambda x: 'Fast'

profit_by_speed_groups=orders.groupby('speed_group')['Profit'].mean
profit_by_speed_groups
```

Out[191…

speed_group	Avg Profit
Fast	28.769
Slow	28.529

Summary. Part 7

There are no material regional differences. Operational performance is stable across Central, East, South, and West. Deliverly speed is driven by shipping mode rather than geography There is no meaningful linear relationship between delivery time and profit and delivery speed does not materially impact profitability. Premium shipping modes generate slightly higher profit per order, but the differences are moderate. Operational delays do not appear to be a major driver of returns.

Part 8. Cohort & Retention Analysis

Objective

Understand customer retention dynamics and revenue longevity.

Tasks

1. Define acquisition cohort
2. Build retention matrix
3. Calculate the Revenue Retention
4. Find a Cohort cumulative profit

Acquisition cohort

In [209…
```
orders['Order_Period'] = orders['Order Date'].dt.to_period('M')
```

In [210…
```
orders['Cohort_Period'] = (
    orders
    .groupby('Customer Name')['Order_Period']
    .transform('min'))
```

In [210…
```
orders['Cohort_Index'] = (
    (orders['Order_Period'].dt.year - orders['Cohort_Period'].dt.ye
    +
    (orders['Order_Period'].dt.month - orders['Cohort_Period'].dt.m
```

In [210…
```
orders[['Customer Name',
        'Order Date',
        'Order_Period',
        'Cohort_Period',
        'Cohort_Index']].sort_values(['Customer Name','Order Date']
```

Out[210…

	Customer Name	Order Date	Order_Period	Cohort_Period	Cohort_Index
119	Aaron Bergman	2022-02-18	2022-02	2022-02	0
161	Aaron Bergman	2022-03-07	2022-03	2022-02	1
164	Aaron Bergman	2022-03-07	2022-03	2022-02	1
167	Aaron Bergman	2022-03-07	2022-03	2022-02	1
6175	Aaron Bergman	2024-11-10	2024-11	2022-02	33
...	...	...	...	...	...
4546	Zuschuss Donatelli	2024-04-03	2024-04	2022-08	20
4723	Zuschuss Donatelli	2024-05-05	2024-05	2022-08	21
5174	Zuschuss Donatelli	2024-07-08	2024-07	2022-08	23
5177	Zuschuss Donatelli	2024-07-08	2024-07	2022-08	23
7856	Zuschuss Donatelli	2025-06-11	2025-06	2022-08	34

10194 rows × 5 columns

Retention Matrix

In [212…

```
cohort_counts = (
    orders
    .groupby(['Cohort_Period', 'Cohort_Index'])['Customer Name']
    .nunique()
    .to_frame('Active Customers'))
cohort_counts
```

Out[212…

Cohort_Period	Cohort_Index	Active Customers
2022-01	**0**	33
	1	3
	3	2
	4	2
	6	2
...	**...**	...
2025-07	**0**	2
	5	1
2025-09	**0**	1
2025-10	**0**	2
2025-11	**0**	1

893 rows × 1 columns

In [212…

```
cohort_matrix = cohort_counts['Active Customers'].unstack(fill_valu
cohort_matrix
```

Out[212…

Cohort_Index	0	1	2	3	4	5	6	7	8	9	...	38	39	40	41	42
Cohort_Period																
2022-01	33	3	0	2	2	0	2	4	5	3	...	6	4	5	4	1
2022-02	24	4	2	1	0	2	2	3	3	4	...	2	2	5	3	3
2022-03	66	5	2	8	7	0	8	5	8	6	...	13	10	12	7	20
2022-04	55	6	2	4	3	7	6	9	8	1	...	5	7	7	14	8
2022-05	57	5	5	4	11	2	12	10	4	3	...	7	6	17	8	18
2022-06	49	1	2	5	2	9	9	1	4	6	...	3	13	7	14	12
2022-07	44	6	5	0	5	8	2	0	8	2	...	11	10	13	9	0
2022-08	52	9	3	11	5	3	4	2	4	4	...	13	13	12	0	0
2022-09	68	9	9	12	1	2	9	10	8	2	...	15	11	0	0	0
2022-10	43	3	5	1	3	4	6	4	3	5	...	14	0	0	0	0
2022-11	63	18	0	3	5	4	2	6	3	5	...	0	0	0	0	0
2022-12	51	3	4	4	2	3	4	6	5	8	...	0	0	0	0	0
2023-01	7	0	1	0	1	1	2	0	0	1	...	0	0	0	0	0
2023-02	7	0	1	0	0	1	0	2	0	0	...	0	0	0	0	0

2023-03	18	1	1	3	1	1	3	1	3	1	...	0	0	0	0	0
2023-04	10	0	0	0	0	1	2	2	0	0	...	0	0	0	0	0
2023-05	12	0	0	1	2	0	2	2	1	1	...	0	0	0	0	0
2023-06	12	1	1	5	1	3	2	0	1	3	...	0	0	0	0	0
2023-07	8	0	0	2	3	1	0	2	0	0	...	0	0	0	0	0
2023-08	11	2	1	2	3	0	0	0	0	1	...	0	0	0	0	0
2023-09	15	1	3	1	3	1	3	3	1	0	...	0	0	0	0	0
2023-10	9	0	4	0	0	0	0	1	0	1	...	0	0	0	0	0
2023-11	9	1	1	0	0	0	1	1	2	1	...	0	0	0	0	0
2023-12	13	1	0	1	2	2	0	3	1	3	...	0	0	0	0	0
2024-01	6	1	0	1	0	1	1	1	0	1	...	0	0	0	0	0
2024-02	2	0	1	0	0	0	0	0	1	1	...	0	0	0	0	0
2024-03	7	0	3	0	3	0	1	1	1	1	...	0	0	0	0	0
2024-04	7	2	0	1	1	4	0	1	0	2	...	0	0	0	0	0
2024-05	7	0	1	2	1	1	2	1	2	1	...	0	0	0	0	0
2024-06	8	1	2	3	1	1	1	1	0	2	...	0	0	0	0	0
2024-07	2	0	0	0	0	0	0	0	0	1	...	0	0	0	0	0
2024-08	4	0	0	0	0	1	0	1	2	2	...	0	0	0	0	0
2024-09	1	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0
2024-10	1	0	1	0	0	0	0	0	0	0	...	0	0	0	0	0
2024-11	6	0	1	0	1	1	0	0	0	1	...	0	0	0	0	0
2024-12	2	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0
2025-03	3	0	0	1	1	1	0	1	0	0	...	0	0	0	0	0
2025-04	1	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0
2025-06	1	0	0	0	0	0	1	0	0	0	...	0	0	0	0	0
2025-07	2	0	0	0	0	1	0	0	0	0	...	0	0	0	0	0
2025-09	1	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0
2025-10	2	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0
2025-11	1	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0

43 rows × 48 columns

In [213…

```
cohort_sizes = cohort_matrix[0]
retention = (cohort_matrix.div(cohort_sizes, axis=0)*100).round(1)
```

```
retention
```

Out[213…

Cohort_Index Cohort_Period	0	1	2	3	4	5	6	7	8	9
2022-01	100.0	9.1	0.0	6.1	6.1	0.0	6.1	12.1	15.2	9.1
2022-02	100.0	16.7	8.3	4.2	0.0	8.3	8.3	12.5	12.5	16.7
2022-03	100.0	7.6	3.0	12.1	10.6	0.0	12.1	7.6	12.1	9.1
2022-04	100.0	10.9	3.6	7.3	5.5	12.7	10.9	16.4	14.5	1.8
2022-05	100.0	8.8	8.8	7.0	19.3	3.5	21.1	17.5	7.0	5.3
2022-06	100.0	2.0	4.1	10.2	4.1	18.4	18.4	2.0	8.2	12.2
2022-07	100.0	13.6	11.4	0.0	11.4	18.2	4.5	0.0	18.2	4.5
2022-08	100.0	17.3	5.8	21.2	9.6	5.8	7.7	3.8	7.7	7.7
2022-09	100.0	13.2	13.2	17.6	1.5	2.9	13.2	14.7	11.8	2.9
2022-10	100.0	7.0	11.6	2.3	7.0	9.3	14.0	9.3	7.0	11.6
2022-11	100.0	28.6	0.0	4.8	7.9	6.3	3.2	9.5	4.8	7.9
2022-12	100.0	5.9	7.8	7.8	3.9	5.9	7.8	11.8	9.8	15.7
2023-01	100.0	0.0	14.3	0.0	14.3	14.3	28.6	0.0	0.0	14.3
2023-02	100.0	0.0	14.3	0.0	0.0	14.3	0.0	28.6	0.0	0.0
2023-03	100.0	5.6	5.6	16.7	5.6	5.6	16.7	5.6	16.7	5.6
2023-04	100.0	0.0	0.0	0.0	0.0	10.0	20.0	20.0	0.0	0.0
2023-05	100.0	0.0	0.0	8.3	16.7	0.0	16.7	16.7	8.3	8.3
2023-06	100.0	8.3	8.3	41.7	8.3	25.0	16.7	0.0	8.3	25.0
2023-07	100.0	0.0	0.0	25.0	37.5	12.5	0.0	25.0	0.0	0.0
2023-08	100.0	18.2	9.1	18.2	27.3	0.0	0.0	0.0	0.0	9.1
2023-09	100.0	6.7	20.0	6.7	20.0	6.7	20.0	20.0	6.7	0.0
2023-10	100.0	0.0	44.4	0.0	0.0	0.0	0.0	11.1	0.0	11.1
2023-11	100.0	11.1	11.1	0.0	0.0	0.0	11.1	11.1	22.2	11.1
2023-12	100.0	7.7	0.0	7.7	15.4	15.4	0.0	23.1	7.7	23.1
2024-01	100.0	16.7	0.0	16.7	0.0	16.7	16.7	16.7	0.0	16.7
2024-02	100.0	0.0	50.0	0.0	0.0	0.0	0.0	0.0	50.0	50.0
2024-03	100.0	0.0	42.9	0.0	42.9	0.0	14.3	14.3	14.3	14.3
2024-04	100.0	28.6	0.0	14.3	14.3	57.1	0.0	14.3	0.0	28.6
2024-05	100.0	0.0	14.3	28.6	14.3	14.3	28.6	14.3	28.6	14.3
2024-06	100.0	12.5	25.0	37.5	12.5	12.5	12.5	12.5	0.0	25.0

2024-07	100.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	50.0
2024-08	100.0	0.0	0.0	0.0	0.0	25.0	0.0	25.0	50.0	50.0
2024-09	100.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
2024-10	100.0	0.0	100.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
2024-11	100.0	0.0	16.7	0.0	16.7	16.7	0.0	0.0	0.0	16.7
2024-12	100.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
2025-03	100.0	0.0	0.0	33.3	33.3	33.3	0.0	33.3	0.0	0.0
2025-04	100.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
2025-06	100.0	0.0	0.0	0.0	0.0	0.0	100.0	0.0	0.0	0.0
2025-07	100.0	0.0	0.0	0.0	0.0	50.0	0.0	0.0	0.0	0.0
2025-09	100.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
2025-10	100.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
2025-11	100.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0

43 rows × 48 columns

In [219…
```
retention_first = retention.head(12).iloc[:, :12]
```

In [219…
```
retention_first.style.background_gradient(cmap='Blues', vmin=0, vma
```

Out[219…

Cohort_Index	**0**	**1**	**2**	**3**	**4**	**5**	**6**	**7**	
Cohort_Period									
2022-01	100.0%	9.1%	0.0%	6.1%	6.1%	0.0%	6.1%	12.1%	1
2022-02	100.0%	16.7%	8.3%	4.2%	0.0%	8.3%	8.3%	12.5%	1
2022-03	100.0%	7.6%	3.0%	12.1%	10.6%	0.0%	12.1%	7.6%	1
2022-04	100.0%	10.9%	3.6%	7.3%	5.5%	12.7%	10.9%	16.4%	1
2022-05	100.0%	8.8%	8.8%	7.0%	19.3%	3.5%	21.1%	17.5%	
2022-06	100.0%	2.0%	4.1%	10.2%	4.1%	18.4%	18.4%	2.0%	
2022-07	100.0%	13.6%	11.4%	0.0%	11.4%	18.2%	4.5%	0.0%	1
2022-08	100.0%	17.3%	5.8%	21.2%	9.6%	5.8%	7.7%	3.8%	
2022-09	100.0%	13.2%	13.2%	17.6%	1.5%	2.9%	13.2%	14.7%	1
2022-10	100.0%	7.0%	11.6%	2.3%	7.0%	9.3%	14.0%	9.3%	
2022-11	100.0%	28.6%	0.0%	4.8%	7.9%	6.3%	3.2%	9.5%	
2022-12	100.0%	5.9%	7.8%	7.8%	3.9%	5.9%	7.8%	11.8%	

Revenue Retention

In [217…
```
cohort_revenue = (orders.groupby(['Cohort_Period', 'Cohort_Index'])
```

Out[217…

Cohort_Index	0	1	2	3	4	
Cohort_Period						
2022-01	14518.0550	225.232	0.000	1472.3820	47.7500	
2022-02	4294.6600	634.310	596.252	576.2340	0.0000	69
2022-03	56299.5990	1595.686	2543.338	3298.7236	3101.2180	
2022-04	24631.0250	2410.624	875.455	2325.4680	454.1520	310
2022-05	20741.8210	1685.128	1862.666	653.2600	8755.0190	46
2022-06	28810.1730	351.216	31.000	1563.7820	104.1520	548
2022-07	25492.1410	2188.379	1044.661	0.0000	1319.1340	425
2022-08	23643.5635	8132.190	272.422	3807.4400	1244.3860	441
2022-09	43601.8548	1882.084	3490.648	7936.0065	31.5380	18
2022-10	21789.9410	563.722	964.936	10.3680	1100.3030	53
2022-11	31895.4987	5487.992	0.000	2563.3640	3601.7220	127
2022-12	21228.9210	3130.160	218.350	1208.8880	1070.4840	89
2023-01	3031.9876	0.000	16.776	0.0000	1461.1395	93
2023-02	1882.9840	0.000	330.150	0.0000	0.0000	17
2023-03	12696.0686	485.784	570.492	2229.2630	860.9160	120
2023-04	2193.8900	0.000	0.000	0.0000	0.0000	
2023-05	5916.3900	0.000	0.000	64.7500	1281.2620	
2023-06	3250.0980	2048.856	310.364	4378.1300	299.9000	17
2023-07	4155.3190	0.000	0.000	869.5500	1370.4820	77
2023-08	6815.8020	1206.586	139.424	3053.8660	302.6460	
2023-09	4971.3840	77.880	1014.638	6.6880	985.1880	12
2023-10	6533.0965	0.000	982.118	0.0000	0.0000	
2023-11	1611.3810	5.760	59.990	0.0000	0.0000	
2023-12	7468.8660	316.000	0.000	30.3360	114.0100	214
2024-01	2855.2500	623.960	0.000	41.5680	0.0000	2
2024-02	46.5440	0.000	88.040	0.0000	0.0000	
2024-03	1451.0720	0.000	5090.168	0.0000	227.3040	
2024-04	2839.4940	115.385	0.000	292.7960	121.0920	91

2024-05	12542.8230	0.000	3136.868	1625.2400	21.8800	10
2024-06	1812.2680	911.256	3888.897	434.0040	54.7920	148
2024-07	109.6300	0.000	0.000	0.0000	0.0000	
2024-08	1985.2140	0.000	0.000	0.0000	0.0000	123
2024-09	9.9900	0.000	0.000	0.0000	0.0000	
2024-10	98.1600	0.000	2003.520	0.0000	0.0000	
2024-11	584.8400	0.000	1370.535	0.0000	296.0220	24
2024-12	2674.3300	0.000	0.000	0.0000	0.0000	
2025-03	485.0500	0.000	0.000	6.3700	558.8100	21
2025-04	16.7390	0.000	0.000	0.0000	0.0000	
2025-06	47.9760	0.000	0.000	0.0000	0.0000	
2025-07	1370.0680	0.000	0.000	0.0000	0.0000	23
2025-09	1038.2600	0.000	0.000	0.0000	0.0000	
2025-10	1593.5280	0.000	0.000	0.0000	0.0000	
2025-11	1205.5840	0.000	0.000	0.0000	0.0000	

43 rows × 48 columns

In [220…
```
cohort_revenue_size = cohort_revenue[0]
```

In [220…
```
revenue_retention = (cohort_revenue.div(cohort_revenue_size, axis=0
```

In [222…
```
revenue_retention_first=revenue_retention.head(12).iloc[:, :12]
```

In [222…
```
revenue_retention_first.style.background_gradient(cmap='Greens', vm
```

Out[222…

Cohort_Index	0	1	2	3	4	5	6	7
Cohort_Period								
2022-01	100.0%	1.6%	0.0%	10.1%	0.3%	0.0%	0.8%	6.3%
2022-02	100.0%	14.8%	13.9%	13.4%	0.0%	16.2%	24.2%	277.3%
2022-03	100.0%	2.8%	4.5%	5.9%	5.5%	0.0%	5.2%	3.3%
2022-04	100.0%	9.8%	3.6%	9.4%	1.8%	12.6%	5.1%	27.5%
2022-05	100.0%	8.1%	9.0%	3.1%	42.2%	2.2%	48.3%	34.5%
2022-06	100.0%	1.2%	0.1%	5.4%	0.4%	19.1%	11.5%	0.2%
2022-07	100.0%	8.6%	4.1%	0.0%	5.2%	16.7%	10.8%	0.0%
2022-08	100.0%	34.4%	1.2%	16.1%	5.3%	18.7%	10.9%	0.7%
2022-09	100.0%	4.3%	8.0%	18.2%	0.1%	0.4%	6.5%	4.9%
2022-10	100.0%	2.6%	4.4%	0.0%	5.0%	24.4%	13.1%	6.3%
2022-11	100.0%	17.2%	0.0%	8.0%	11.3%	4.0%	0.4%	4.9%
2022-12	100.0%	14.7%	1.0%	5.7%	5.0%	4.2%	5.1%	19.5%

Cohort Cumulative Profit

In [222…
```
cohort_profit = (orders.groupby(['Cohort_Period', 'Cohort_Index'])[
```

In [223…
```
cohort_cum_profit = cohort_profit.cumsum(axis=1)
```

In [223…
```
cohort_cum_profit_first = cohort_cum_profit.head(12).iloc[:, :12]
```

In [224…
```
cohort_cum_profit_first.style.background_gradient(cmap='Oranges', v
```

Out[224…

Cohort_Index	0	1	2	3	4	5	6	7	
Cohort_Period									
2022-01	2,539	2,575	2,575	2,336	2,353	2,353	2,388	2,514	2
2022-02	827	875	1,057	1,136	1,136	1,172	1,539	7,052	7
2022-03	645	412	526	895	1,296	1,296	1,840	1,885	2
2022-04	3,779	4,021	4,205	4,723	4,878	4,003	4,161	5,589	6
2022-05	2,744	3,210	3,463	3,340	2,063	2,131	3,211	3,935	2
2022-06	3,981	3,985	4,001	4,452	4,487	4,276	4,478	4,499	4
2022-07	-2,090	-1,469	-1,368	-1,368	-1,588	-446	7	7	1
2022-08	4,606	6,157	6,235	6,868	6,564	4,745	5,041	5,101	5
2022-09	2,205	2,394	2,571	2,946	2,954	2,910	2,886	3,534	4
2022-10	3,143	3,345	3,422	3,425	3,487	5,640	5,694	5,769	5
2022-11	2,595	3,251	3,251	4,531	5,272	5,513	5,536	5,975	6
2022-12	2,284	2,409	2,466	2,461	2,307	2,548	2,331	2,857	2

Summary. Part 8

Cohort analysis reveals low short-term retention, with only 10–15% of customers returning in the first month. Retention fluctuates rather than declining steadily, reflecting irregular purchasing behavior. A modest long-tail effect persists, with 7–15% of customers active up to 12 months. Q4 cohorts show slightly stronger early retention, suggesting seasonality or campaign impact. Revenue retention is more volatile than customer retention. Early-month revenue return remains low, but some cohorts generate significant spikes later, occasionally exceeding 100% of initial revenue. This indicates that a small share of returning customers drives disproportionate revenue through high-value purchases. Most cohorts are profitable from month one, though profit growth is uneven and episodic. Some cohorts require several months to reach breakeven. Overall, cumulative profit increases gradually over 6–12 months, showing that lifetime value materializes over time. Overall the business operates on an episodic retail model with weak short-term retention but stable long-term value driven by occasional high-margin repeat purchases.

Part 9. Regional Manager Performance

Objective

Assess regional manager effectiveness by analyzing sales, profit dynamics, and risk patterns across regions.

Tasks

1. Provide Profit Stability Analysis
2. Conduct Loss & Risk Analysis
3. Find Revenue Structure & Concentration

4. Calculate Profit Contribution

Profit Stability Analysis

In [226…
```
manager_data = (orders.merge(people, on='Region', how='left'))
manager_data['Year_Month'] = manager_data['Order Date'].dt.to_perio
monthly_profit = (manager_data.groupby(['Regional Manager', 'Year_M
```

In [227…
```
manager_stability = (monthly_profit.groupby('Regional Manager')['Pr
        .agg(
            avg_monthly_profit='mean',
            profit_std='std',
            min_monthly_profit='min',
            max_monthly_profit='max',
            active_months='count')
        .reset_index()).round(2)
```

In [227…
```
manager_stability
```

Out[227…

	Regional Manager	avg_monthly_profit	profit_std	min_monthly_profit	max_mont
0	Chuck Magee	1976.735	2481.826	-4842.242	
1	Fred Suzuki	973.946	1269.510	-2243.235	
2	Roxanne Rodriguez	830.527	2198.093	-4055.334	
3	Sadie Pawthorne	2308.309	1875.580	-1723.215	

In [228…
```
manager_stability['volatility_ratio'] = (manager_stability['profit_
```

In [228…
```
manager_stability
```

Out[228…

	Regional Manager	avg_monthly_profit	profit_std	min_monthly_profit	max_mont
0	Chuck Magee	1976.73	2481.83	-4842.24	
1	Fred Suzuki	973.95	1269.51	-2243.24	
2	Roxanne Rodriguez	830.53	2198.09	-4055.33	
3	Sadie Pawthorne	2308.31	1875.58	-1723.21	

Loss & Risk Analysis

In [230…
```
manager_data['is_loss'] = manager_data['Profit'] < 0
```

In [231…
```
loss_metrics = (
    manager_data.groupby('Regional Manager').agg(
            total_orders=('Order ID', 'count'),
            loss_orders=('is_loss', 'sum'),
            total_profit=('Profit', 'sum'),
            total_loss_amount=('Profit', lambda x: x[x < 0].sum()))
```

In [231…
```
loss_metrics['loss_rate_%'] = (
    loss_metrics['loss_orders'] /
    loss_metrics['total_orders'] * 100
).round(1)
```

In [231…
```
loss_metrics=loss_metrics.sort_values(['loss_rate_%', 'total_loss_a
loss_metrics
```

Out[231…

	Regional Manager	total_orders	loss_orders	total_profit	total_loss_amount	los
2	Roxanne Rodriguez	2335	747	39865.3070	-56371.1035	
0	Chuck Magee	2986	574	94883.2603	-50187.8375	
1	Fred Suzuki	1620	259	46749.4303	-27504.8323	
3	Sadie Pawthorne	3253	321	110798.8170	-22975.1569	

Revenue Structure & Concentration

In [232…
```
manager_category_sales = (manager_data.groupby(['Regional Manager',
```

In [233…
```
manager_total_sales = (manager_category_sales.groupby('Regional Man
```

In [233…
```
manager_category_sales = (manager_category_sales.merge(manager_tota

manager_category_sales['category_share_%'] = (manager_category_sale
manager_category_sales['category_share_%']=manager_category_sales['
```

In [233…
```
top_category = (manager_category_sales.sort_values(['Regional Manag
        .drop_duplicates('Regional Manager')
        .reset_index(drop=True))
```

In [234…
```
top_category
```

Out[234…

	Regional Manager	Category	Sales	total_sales	category_share_%
0	Chuck Magee	Technology	267938.0710	691828.1680	38.7
1	Fred Suzuki	Technology	148771.9080	391721.9050	38.0
2	Roxanne Rodriguez	Technology	170416.3120	503170.6728	33.9
3	Sadie Pawthorne	Furniture	260679.7295	739813.6085	35.2

Profit Contribution

In [236…

```
manager_profit = (manager_data.groupby('Regional Manager')['Profit'
```

In [236…

```
total_company_profit = manager_profit['Profit'].sum()
```

In [237…

```
manager_profit['profit_share_%'] = (manager_profit['Profit'] /total_
```

In [237…

```
manager_profit=manager_profit.sort_values('profit_share_%',ascendin
manager_profit
```

Out[237…

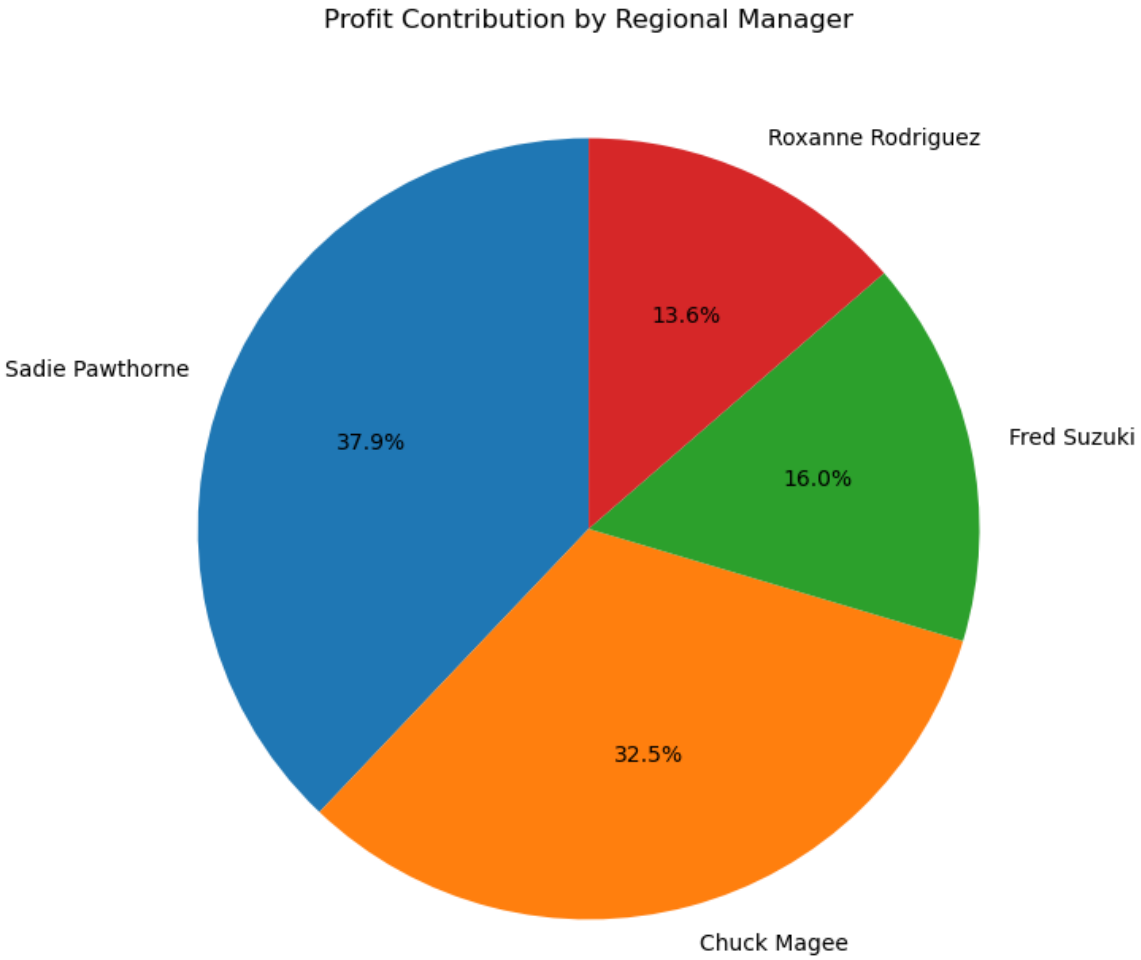
	Regional Manager	Profit	profit_share_%
3	Sadie Pawthorne	110798.8	37.9
0	Chuck Magee	94883.3	32.5
1	Fred Suzuki	46749.4	16.0
2	Roxanne Rodriguez	39865.3	13.6

In [239…

```
viz_manager_profit= manager_profit.plot(
    kind='pie',
    y= 'Profit',
    labels=manager_profit['Regional Manager'],
    autopct='%1.1f%%',
    startangle=90,
    figsize=(8,8),
    legend=False)
viz_manager_profit.set_title('Profit Contribution by Regional Manag
viz_manager_profit.set_ylabel('')
```

Out[239…

```
Text(0, 0.5, '')
```



Summary. Part 9

Sadie Pawthorne stands out as the most stable and risk-efficient performer, generating the highest average monthly profit ($2,308) with the lowest volatility ratio (0.81). She maintains the lowest loss rate (9.9%) and the smallest total loss impact (–$22,975), while contributing 37.9% of total company profit ($110,799). # Chuck Magee is highly profitable, with an average monthly profit of $1,977 and total profit of $94,883 (32.5% of company profit). However, his volatility ratio (1.26) and 19.2% loss rate indicate moderate instability compared to Sadie. Fred Suzuki operates at a mid-level performance, averaging $974 per month with a volatility ratio of 1.30 and a 16.0% loss rate. His total contribution accounts for 16.0% of overall profit ($46,749), reflecting controlled but limited impact. The majority of company downside risk is driven by Roxanne Rodriguez, who shows the highest volatility (2.65), the highest loss rate (32%), and the largest total loss impact (–$56K). Overall, 70% of the company's profit is generated by two managers (Sadie and Chuck), while downside risk is disproportionately concentrated in one high-volatility manager - Roxanne.

Part 10. Key Metrics Forecast

Objective

Project next year’s Sales and Profit based on historical monthly trends to

estimate expected growth and assess future business performance under a baseline scenario.

Tasks

1. Trend & Growth Analysis
2. Build Baseline Forecast Model
3. Compare Actual vs Forecast

Trend & Growth Analysis

In [242…
```
monthly_ts = (orders.groupby('Year_Month').agg(
            Sales=('Sales', 'sum'),
            Profit=('Profit', 'sum')).sort_index())
monthly_ts.index = monthly_ts.index.to_timestamp()
```

In [243…
```
monthly_ts['Sales_MoM_%'] = (monthly_ts['Sales'].pct_change() * 100

monthly_ts['Profit_MoM_%'] = (monthly_ts['Profit'].pct_change() * 1
```

In [243…
```
monthly_ts['Sales_Rolling_3'] = (monthly_ts['Sales'].rolling(3).mea

monthly_ts['Profit_Rolling_3'] = (monthly_ts['Profit'].rolling(3).m
```

In [243…
```
monthly_ts['Sales_YoY_%'] = (monthly_ts['Sales'].pct_change() * 100

monthly_ts['Profit_YoY_%'] = (monthly_ts['Profit'].pct_change() * 1
```

In [244…
```
years_diff = (monthly_ts.index.max().year - monthly_ts.index.min().

sales_avg_annual_growth = ((monthly_ts['Sales'].iloc[-1] / monthly_

profit_avg_annual_growth = ((monthly_ts['Profit'].iloc[-1] / monthl
```

Baseline Forecast Model

In [244…
```
x=np.arange(len(monthly_ts))
```

In [245…
```
sales_trend = np.poly1d(np.polyfit(x, monthly_ts['Sales'], 1))
```

In [245…
```
profit_trend = np.poly1d(np.polyfit(x, monthly_ts['Profit'], 1))
```

In [245…
```
future_x = np.arange(len(monthly_ts), len(monthly_ts) + 12)

future_dates = pd.date_range(
    start=monthly_ts.index[-1] + pd.offsets.MonthBegin(1),
    periods=12,
    freq='MS'
```

```
)
```

In [245…

```
forecast_df = pd.DataFrame({
    'Date': future_dates,
    'Forecast_Sales': sales_trend(future_x),
    'Forecast_Profit': profit_trend(future_x)
})
```

In [246…

```
forecast_df
```

Out[246…

	Date	Forecast_Sales	Forecast_Profit
0	2026-01-01	70799.191972	9543.821573
1	2026-02-01	71710.609370	9684.813598
2	2026-03-01	72622.026768	9825.805622
3	2026-04-01	73533.444166	9966.797647
4	2026-05-01	74444.861565	10107.789671
5	2026-06-01	75356.278963	10248.781696
6	2026-07-01	76267.696361	10389.773720
7	2026-08-01	77179.113759	10530.765745
8	2026-09-01	78090.531158	10671.757770
9	2026-10-01	79001.948556	10812.749794
10	2026-11-01	79913.365954	10953.741819
11	2026-12-01	80824.783352	11094.733843

In [246…

```
forecast_df = forecast_df.set_index('Date')
```

In [246…

```
combined_sales = pd.concat([monthly_ts['Sales'], forecast_df['Forec

combined_profit = pd.concat([monthly_ts['Profit'], forecast_df['For
```

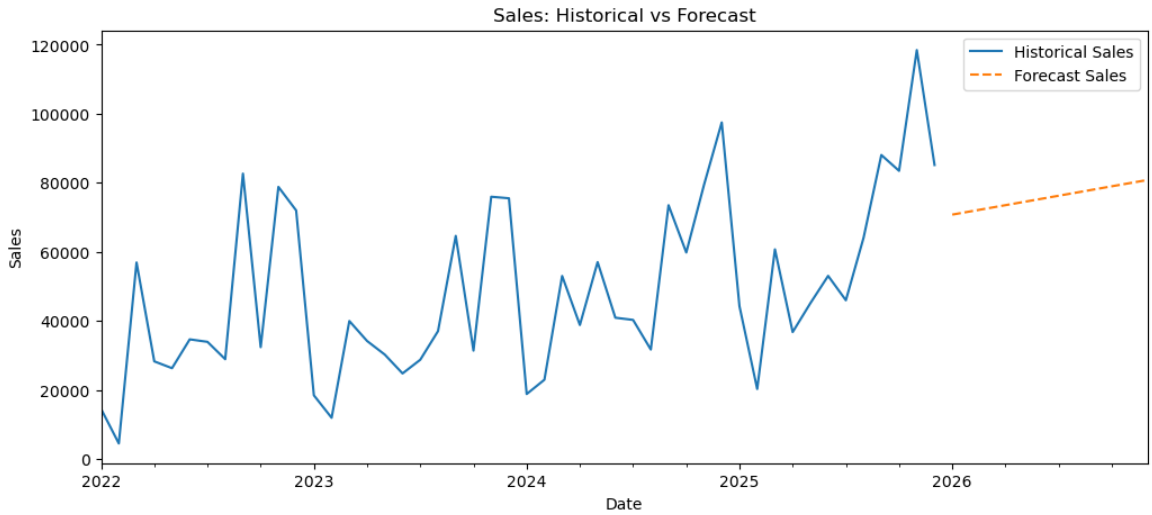
In [246…

```
sales_plot = pd.concat(
    [
        monthly_ts['Sales'],
        forecast_df['Forecast_Sales']
    ],
    axis=1
)

sales_plot.columns = ['Historical Sales', 'Forecast Sales']

ax = sales_plot.plot(
    figsize=(12,5),
    style=['-', '--']
)
```

```
ax.set_title('Sales: Historical vs Forecast')
ax.set_xlabel('Date')
ax.set_ylabel('Sales')
```

Out[246…

```
Text(0, 0.5, 'Sales')
```

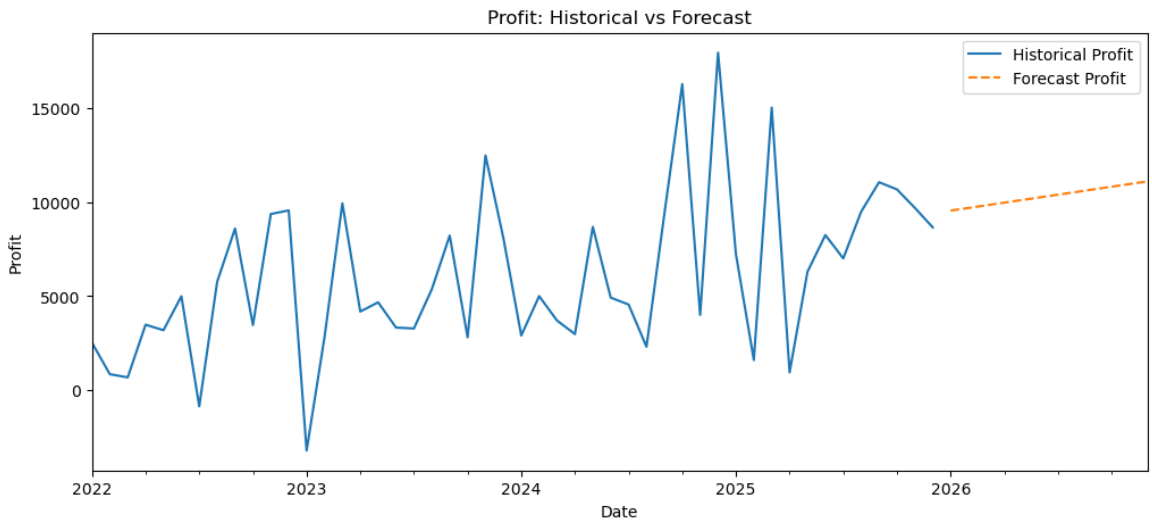


In [247…

```
profit_plot = pd.concat(
    [
        monthly_ts['Profit'],
        forecast_df['Forecast_Profit']
    ],
    axis=1
)

profit_plot.columns = ['Historical Profit', 'Forecast Profit']

ax = profit_plot.plot(
    figsize=(12,5),
    style=['-', '--']
)

ax.set_title('Profit: Historical vs Forecast')
ax.set_xlabel('Date')
ax.set_ylabel('Profit')
```

Out[247…

```
Text(0, 0.5, 'Profit')
```



Compare Actual vs Forecast

In [248…
```
last_year = monthly_ts.index.year.max()

actual_last_year_sales = (monthly_ts[monthly_ts.index.year == last_

actual_last_year_profit = (monthly_ts[monthly_ts.index.year == last
```

In [248…
```
forecast_sales_next_year = forecast_df['Forecast_Sales'].sum().roun
forecast_profit_next_year = forecast_df['Forecast_Profit'].sum().ro
```

In [250…
```
sales_growth_forecast = ((forecast_sales_next_year / actual_last_ye
profit_growth_forecast = ((forecast_profit_next_year / actual_last_
```

In [251…
```
comparison_table = pd.DataFrame({
    'Metric': ['Sales', 'Profit'],
    'Actual Last Year': [actual_last_year_sales,actual_last_year_pr
    'Forecast Next Year': [forecast_sales_next_year,forecast_profit
    'Forecast Growth %': [sales_growth_forecast,profit_growth_forec
```

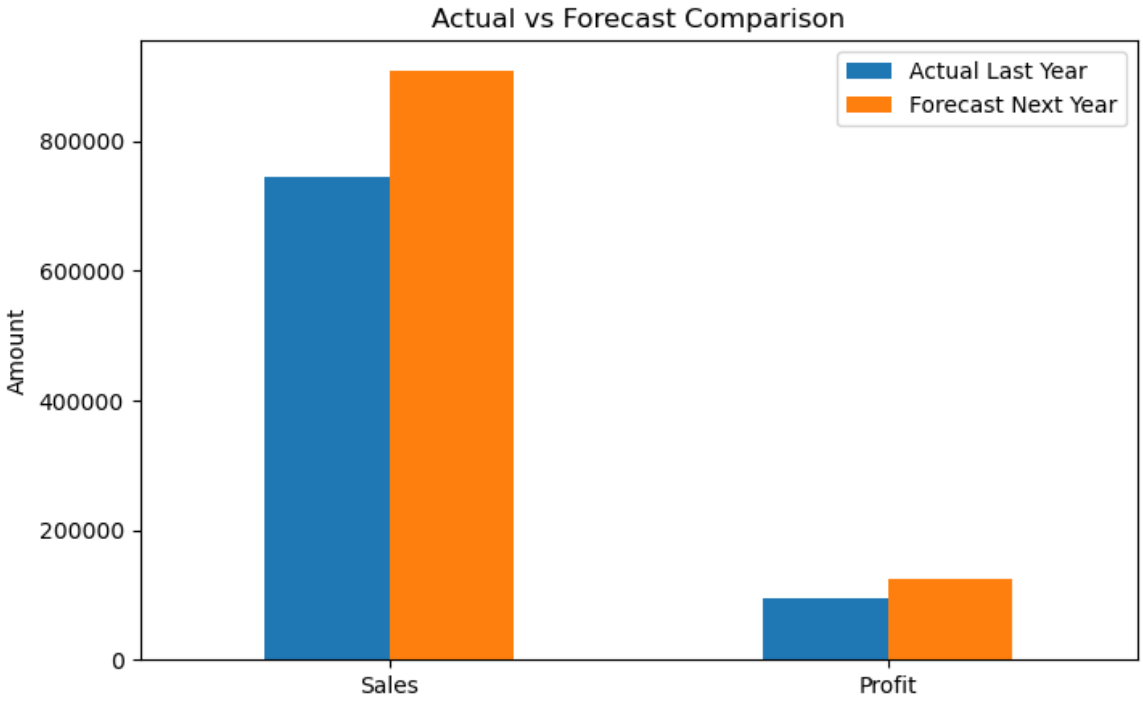
In [251…
```
comparison_table
```

Out[251…

	Metric	Actual Last Year	Forecast Next Year	Forecast Growth %
0	Sales	745567.5	909744.0	22.0
1	Profit	95926.3	123831.0	29.1

In [254…
```
bar_data = comparison_table.set_index('Metric')[[
    'Actual Last Year',
    'Forecast Next Year']]

viz_bar_data = bar_data.plot(
    kind='bar',
    figsize=(8,5)
)

viz_bar_data.set_title('Actual vs Forecast Comparison')
viz_bar_data.set_ylabel('Amount')
viz_bar_data.set_xlabel('')
viz_bar_data.legend(title='')
viz_bar_data.set_xticklabels(
    viz_bar_data.get_xticklabels(),
    rotation=0
)
```

Out[254…
```
[Text(0, 0, 'Sales'), Text(1, 0, 'Profit')]
```



Summary. Part 10

A linear trend projection was applied to last year monthly Sales and Profit to estimate baseline performance for 2026. The forecast indicates steady growth throughout the year: Monthly Sales are projected to increase from 70.8K in January 2026 to 80.8K in December 2026. Monthly Profit is expected to grow from 9.5K to 11.1K over the same period. On an annual basis: Sales are projected to reach 909,744, compared to 745,568 in the last observed year — a +22.0% increase. Profit is projected to reach 123,831, compared to 95,926 — a +29.1% increase. Profit is expected to grow faster than Sales, suggesting potential improvement in operational efficiency or margin expansion under the trend assumption. This forecast represents a baseline linear continuation of historical dynamics. It assumes stable growth without structural changes, seasonality shocks, or external disruptions.